

第七章 查找

7.1 查找的基本概念

1. 查找：在数据集中寻找满足某种条件的数据元素的过程称为查找。
2. 查找表（查找结构）：用于查找的数据集合称为查找表，它由**同一类型**的数据元素（或记录）组成。
3. 关键字：数据元素中唯一标识该元素的某个数据项的值，使用基于关键字的查找，查找结果应该是唯一的。
4. 对查找表的常见操作：
 1. 查找符合条件的数据元素
 2. 插入、删除某个数据元素

只进行查找操作最好使用静态查找表，若需要进行大量插入删除操作可使用动态查找表。

1. 查找长度：在查找运算中，需要对比关键字的次数称为查找长度。
2. 平均查找长度（ASL, Average Search Length）：所有查找过程中进行关键字的比较次数的平均值。

$$ASL = \sum_{i=1}^n P_i C_i$$

学号	姓名	语文	数学	英语
202001	铁柱	99	97	98
202002	狗剩	86	23	90
202003	旺财	43	66	76
202004	愣子	12	16	11
202005	铁柱	9	17	25

只需进行操作① —— 静态查找表

仅关注查找速度即可

订单号	菜品	下单时间	已完成
1	黄焖Jimmy饭	30min	是
2	鱼香Rose	20min	否
3	皮蛋solo粥	15min	是
4	梅菜Crow	10min	否
5	过桥Michelle	2min	否

也要进行操作② —— 动态查找表

除了查找速度，也要关注
插/删操作是否方便实现

7.2 顺序查找和折半查找

7.2.1 顺序查找

1. 顺序查找：又叫“线性查找”，通常用于线性表。
2. 算法思想：从头到尾挨个找（或者从尾到头）。
3. 代码实现：

```

typedef struct {           // 顺序查找的数据结构
    int *elem;             // 动态数组基址
    int TableLen;          // 表长
} SSTable;

int Search_Seq(SSTable ST, int key) {
    int i;
    for (i = 0; i < ST.TableLen && ST.elem[i] != key; ++i);
    // 查找成功，则返回元素下标，否则返回-1
    return i == ST.TableLen ? -1 : i;
}

```

带哨兵方式（优点：不用判断数组是否越界，效率略高）：

```

typedef struct {           // 顺序查找的数据结构
    int *elem;             // 动态数组基址
    int TableLen;          // 表长
} SSTable;

int Search_Seq(SSTable ST, int key) {
    ST.elem[0] = key;
    int i;
    for (i = ST.TableLen; ST.elem[i] != key; --i)
        // 查找成功返回数组下标，否则返回0
        return i;
}

```

查找效率分析：

$$ASL_{\text{成功}} = \frac{1+2+3+\dots+n}{n} = \frac{n+1}{2}$$

$$ASL_{\text{失败}} = n + 1$$

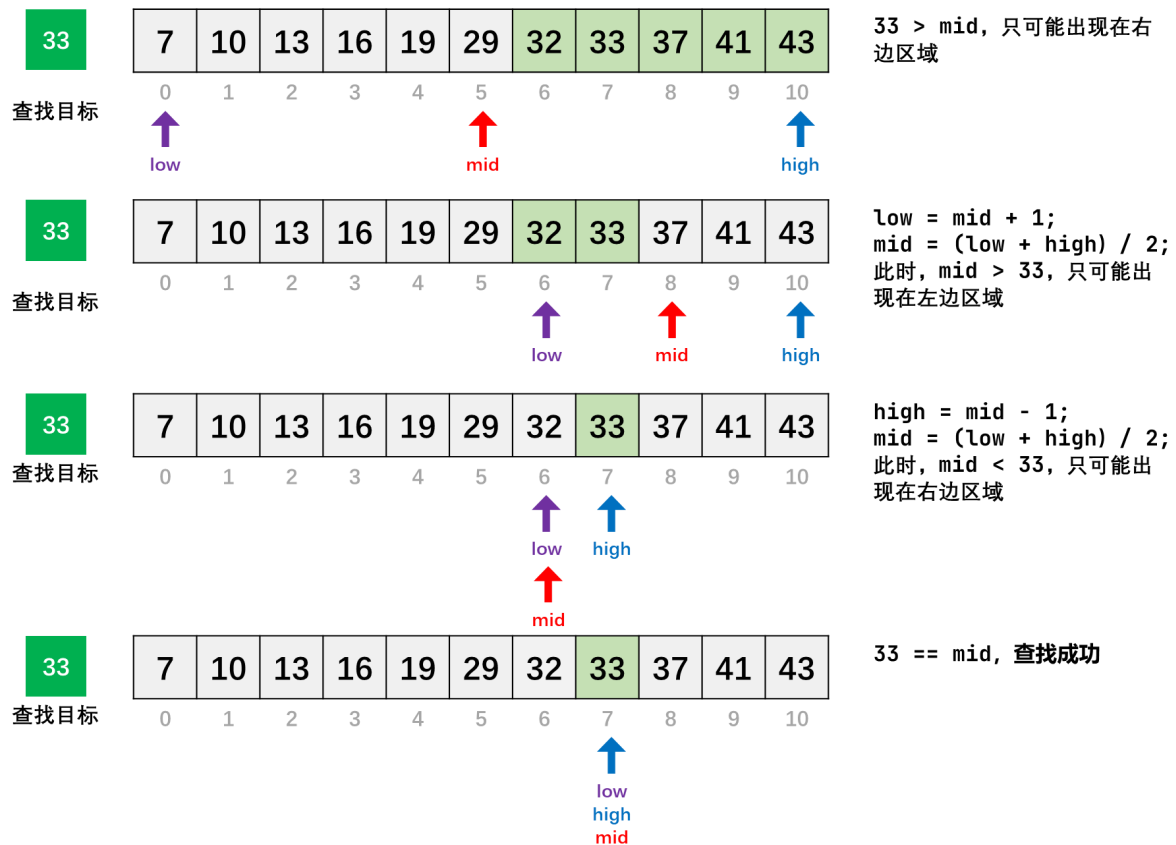
7.2.2 折半查找

折半查找，又称“二分查找”，仅适用于**有序的顺序表**。

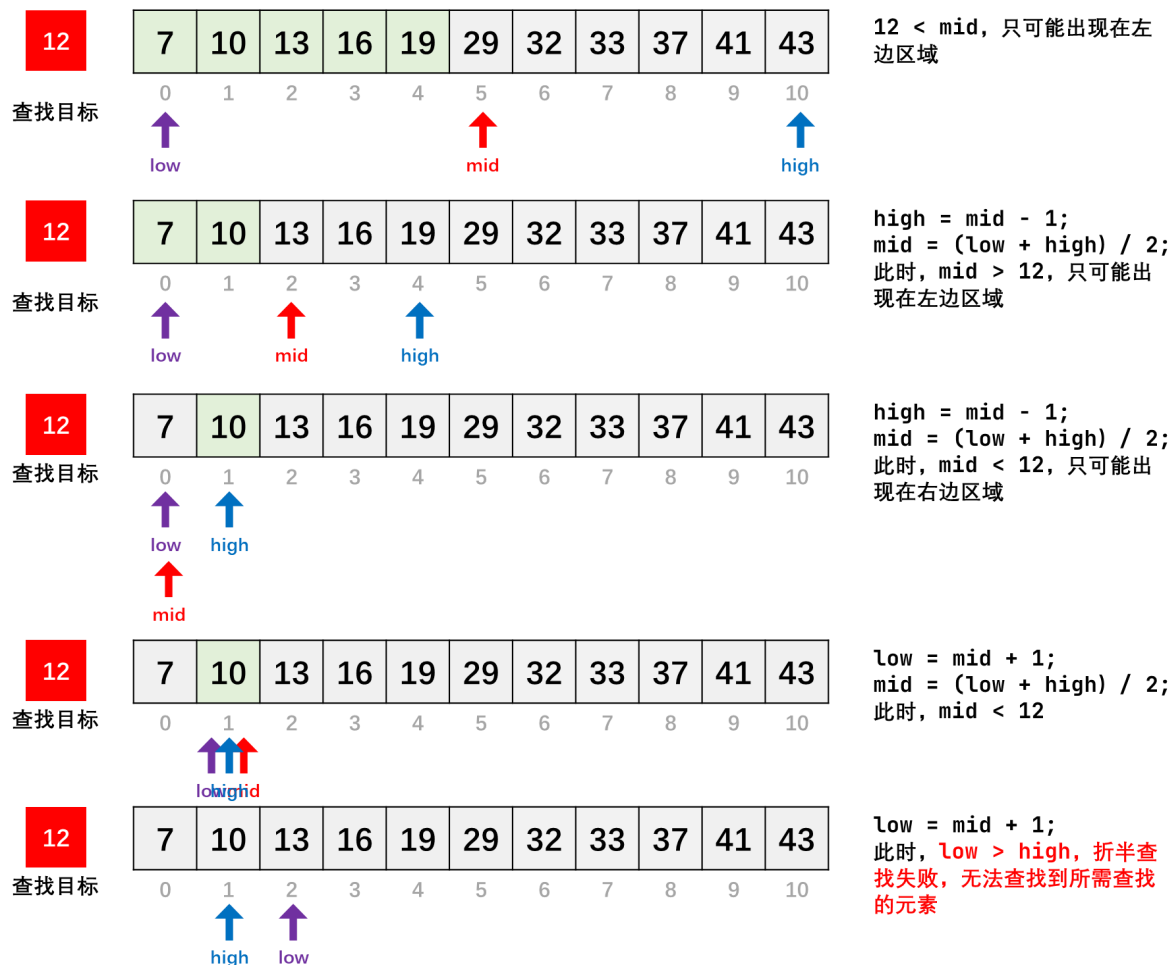
顺序表具有随机访问的特性，而链表没有。

其查找示例如下：

查找成功的情况：



查找失败的情况:



代码实现:

```
typedef struct {
```

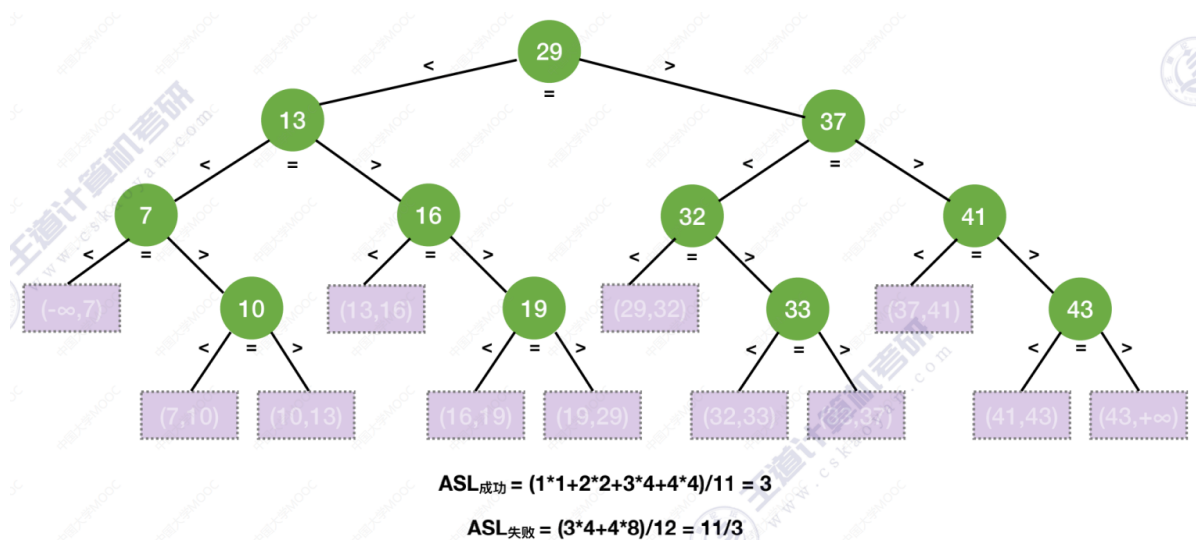
```

ElemType *elem;
int TableLen;
} SSTable;

// 折半查找
int Binary_Search(SSTable L, ElemType key) {
    int low = 0, high = L.TableLen, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (L.elem[mid] == key)
            return mid;
        else if (L.elem[mid] > key)
            high = mid - 1;           //从前半部分继续查找
        else
            low = mid + 1;           //从后半部分继续查找
    }
    return -1;
}

```

查找效率分析



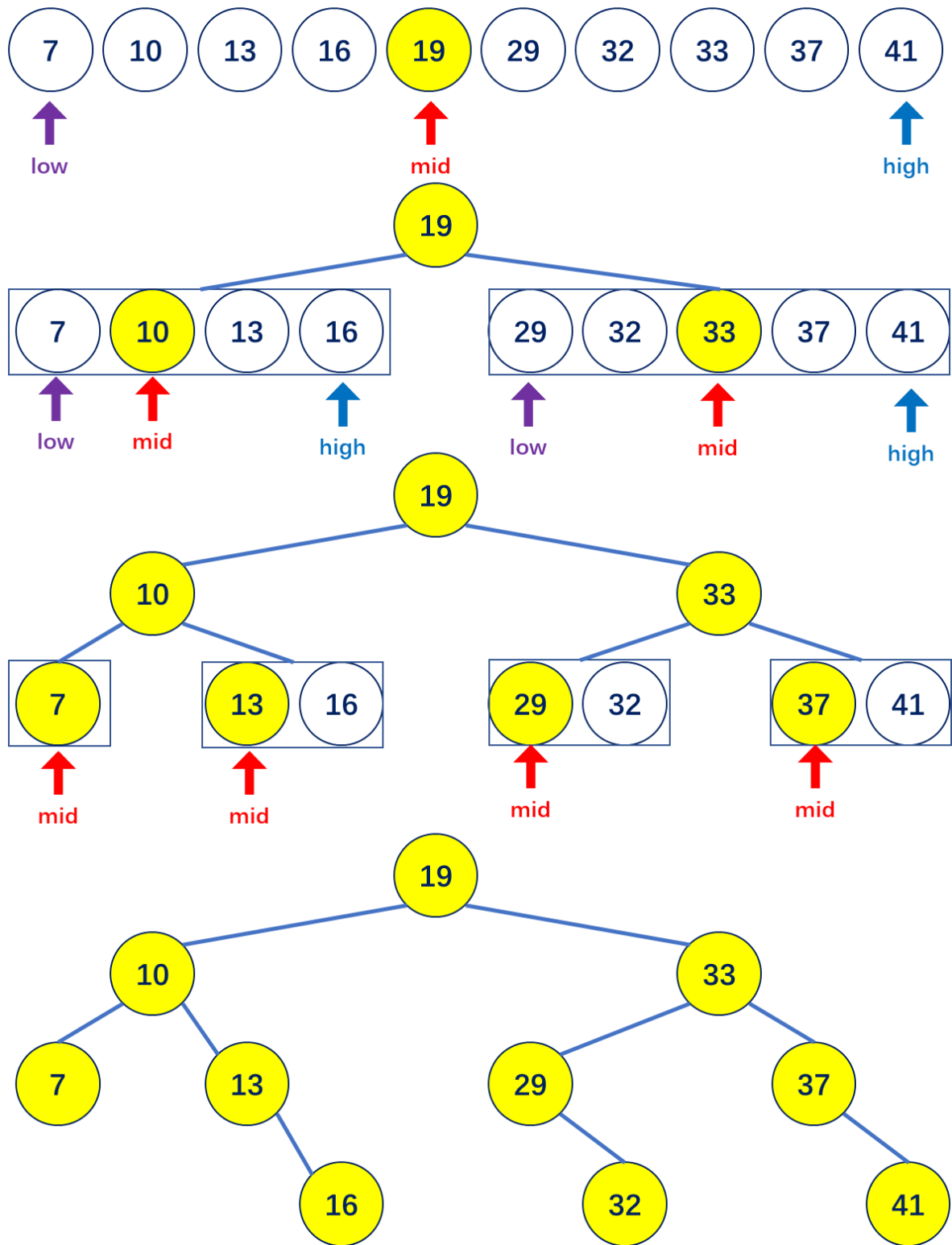
折半查找的过程可以由一棵查找判定树来表示。其构造过程如下：

构造原则：

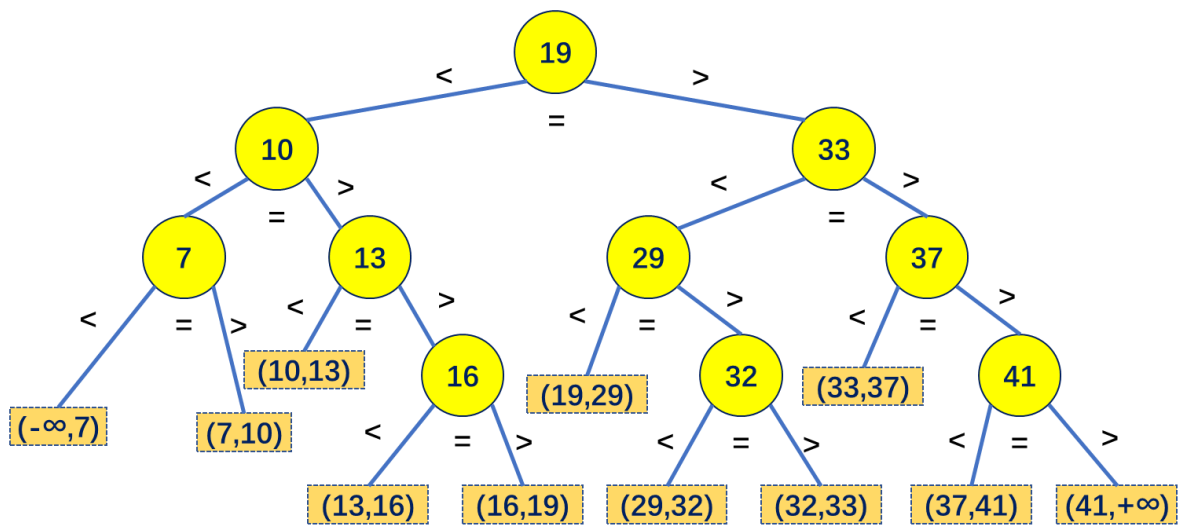
如果当前low和high之间有奇数个元素，则mid分隔后，左右两部分元素个数相等

如果当前low和high之间有偶数个元素，则mid分隔后，左半部分比右半部分少一个元素

折半查找判定树的构造



最终构造完成的折半查找判定树如下：



【注意】

1. 折半查找判定树一定是平衡二叉树
2. 折半查找判定树中，只有最下面一层是不满的，因此，元素个数为 n 时，树高 $h = \lceil \log_2(n+1) \rceil$ (该树高不包括失败结点)，计算方法同完全二叉树。

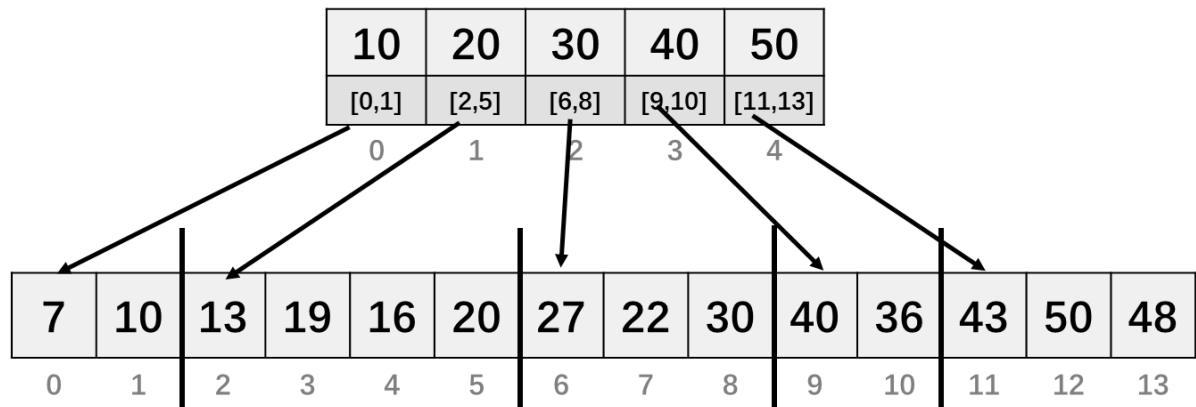
查找成功的 $ASL \leq h$ ，查找失败的 $ASL \leq h$

折半查找的时间复杂度= $O(\log_2 n)$

7.2.3 分块查找

1、算法思想

分块查找使用索引表来保存每个分块的最大关键字和分块存储区间。其特点为块内无序，块间有序。



其结构定义如下：

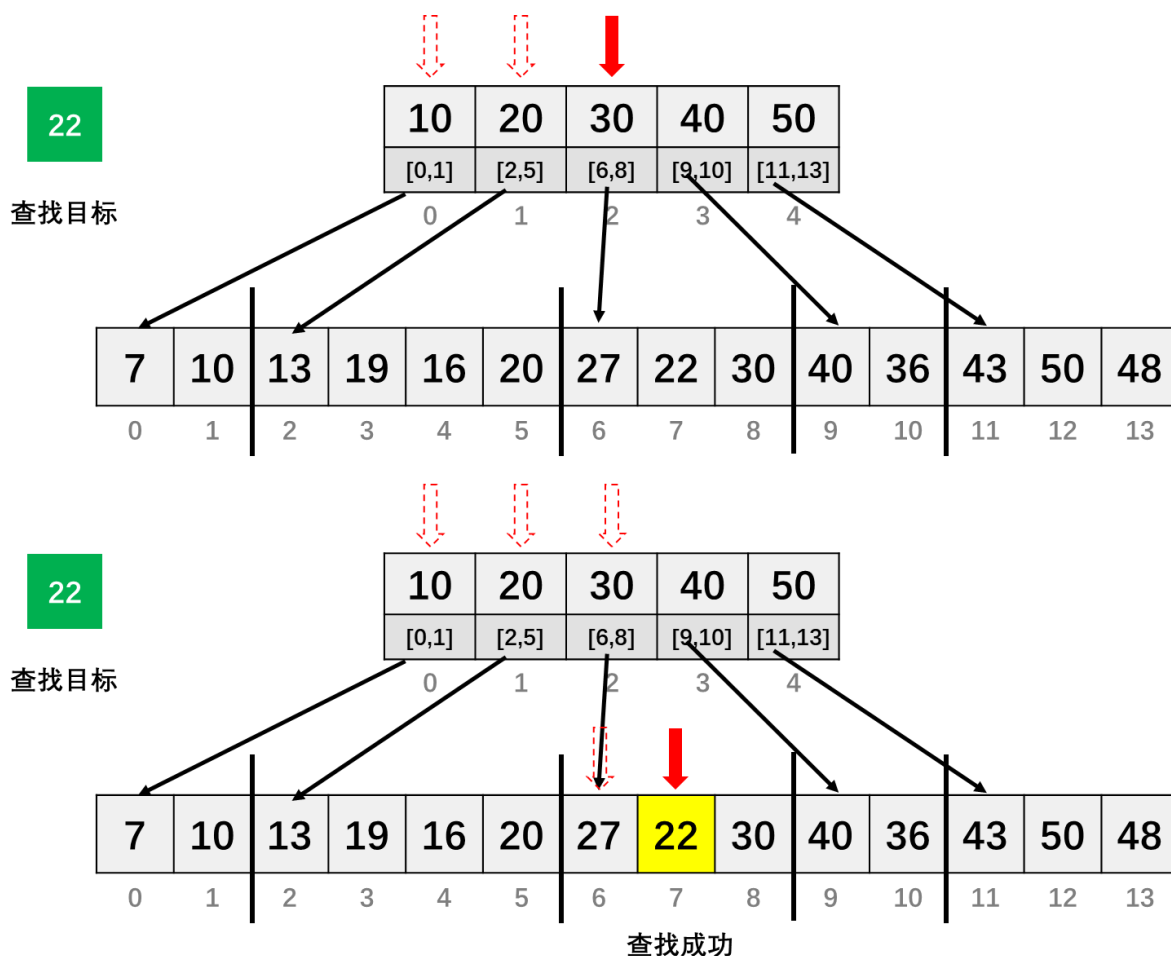
```
// 索引表
typedef struct {
    ElemType maxValue;
    int low, high;
} Index;
// 顺序表存储实际元素
ElemType List[100];
```

其查找流程如下：

在索引表中确定待查记录所属的分块（可使用顺序、折半查找）

【注意】使用折半查找时，若索引表不含目标关键字，则折半查找最终停留在 $low > high$ ，此时，需要在 low 所指向的分块中查找。若 low 超出索引表范围，则查找失败，

在找到的分块内顺序查找（因为块内无序，无法使用折半查找）



分块查找一般不考察代码。

2、算法效率分析

- $ASL = \text{查找索引表的平均查找长度} + \text{查找分块的平均查找长度}$
- 设有 n 个记录，均匀分为 b 块，每块 s 个记录。

◦ 顺序查找索引表：

- $ASL = \frac{b+1}{2} + \frac{s+1}{2}$
- 当 $s = \sqrt{n}$ 时， $ASL_{\text{最小}} = \sqrt{n} + 1$

*这里的 s 为理想块长

◦ 折半查找索引表：

- $ASL = \lceil \log_2(b+1) \rceil + \frac{s+1}{2}$

【注意】对索引表进行折半查找时，若索引表中不包含目标关键字，则折半查找最终停在 $low > high$ ，要在 low 所指分块中查找。

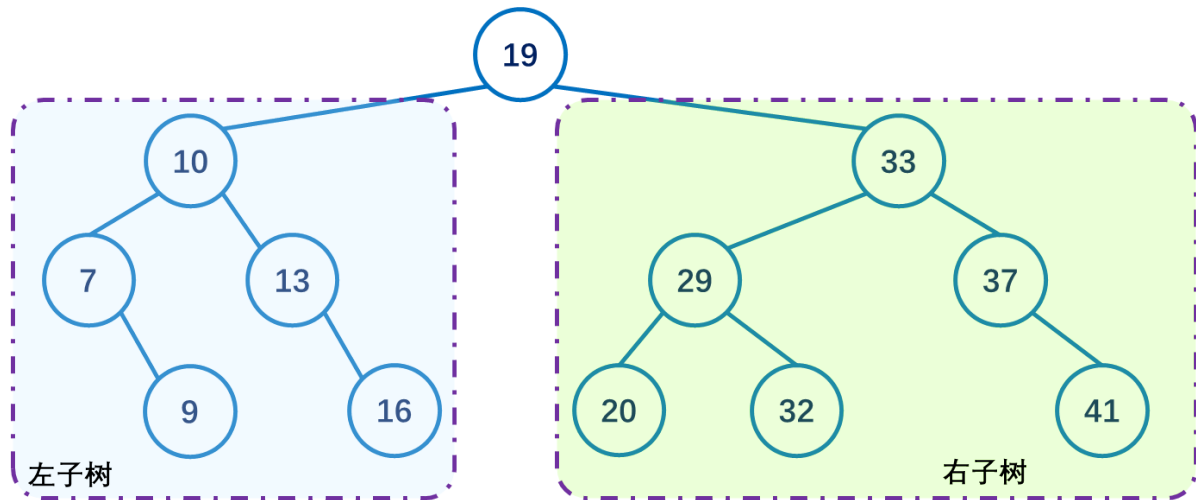
7.3 树形查找

7.3.1 二叉排序树 (BST)

1、定义

二叉排序树（也称二叉查找树）或者是一棵空树，或者是具有下列特性的二叉树：

1. 若左子树非空，则左子树上所有结点的值均小于根结点的值。
2. 若右子树非空，则右子树上所有结点的值均大于根结点的值。
3. 左、右子树也分别是一棵二叉排序树。



左子树节点值 < 根节点值 < 右子树节点值

经过中序遍历，可得到一个**递增的有序序列**。

如上述二叉排序树经过中序遍历得到的序列为：7, 9, 10, 13, 16, 19, 20, 29, 32, 33, 37, 41

其结构定义如下：

```
typedef struct BSTNode {  
    int key;  
    struct BSTNode *lchild, *rchild;  
} BSTNode, *BSTree;
```

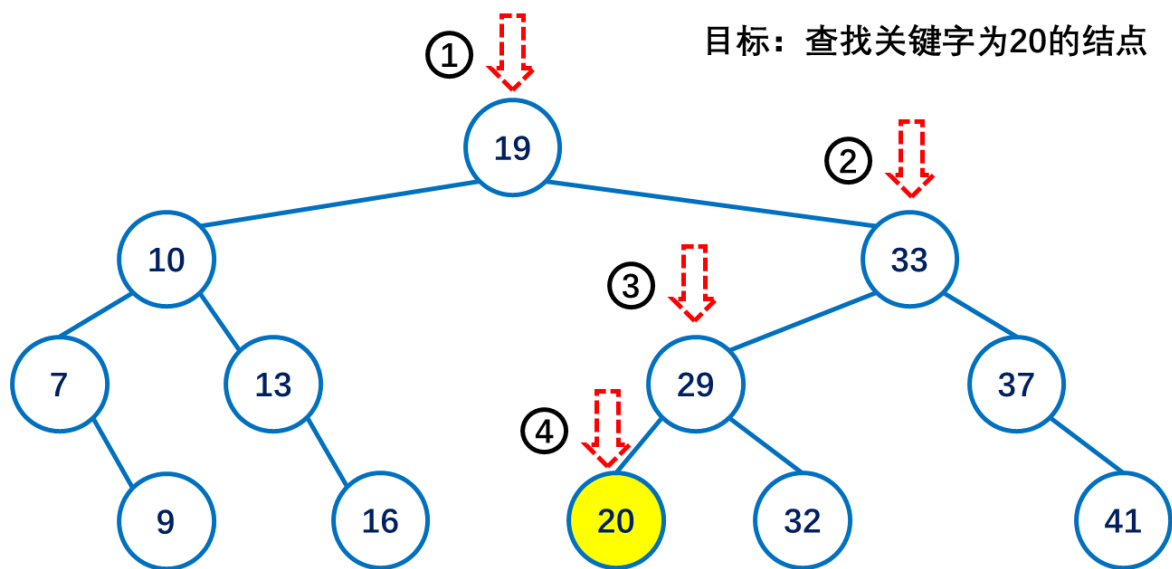
2、二叉排序树的查找

若树非空，目标值与根结点的值比较；若相等，则查找成功；

若小于根结点，则在左子树上查找，否则在右子树上查找。

查找成功，返回结点指针；

查找失败返回NULL。



代码实现：

```

// 非递归写法
BSTNode *BST_Search(BSTree T, int key) {
    while (T != NULL && key != T->key) {    // 若树空或等于根节点值，结束循环
        if (key < T->key)
            T = T->lchild;                    // 根节点值大于关键字的值，在左子树中查找
        else
            T = T->rchild;                    // 根节点值小于关键字的值，在右子树中查找
    }
    return T;
}

// 递归写法
BSTNode *BSTSearch(BSTree T, int key) {
    if (T == NULL)
        return NULL;
    if (key == T->key)
        return T;
    else if (key < T->key)
        return BSTSearch(T->lchild, key);
    else
        return BSTSearch(T->rchild, key);
}
  
```

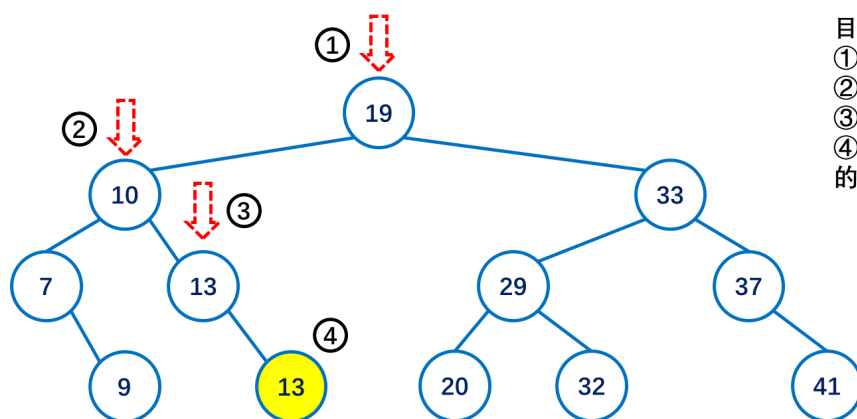
非递归写法最坏空间复杂度： $O(1)$

递归写法最坏空间复杂度： $O(h)$ (h 为树高)

3、二叉排序树的插入

若原二叉排序树为空，则直接插入结点；否则，若关键字 k 小于根结点值，则插入到左子树，若关键字 k 大于根结点值，则插入到右子树。

新插入的结点一定是叶子结点



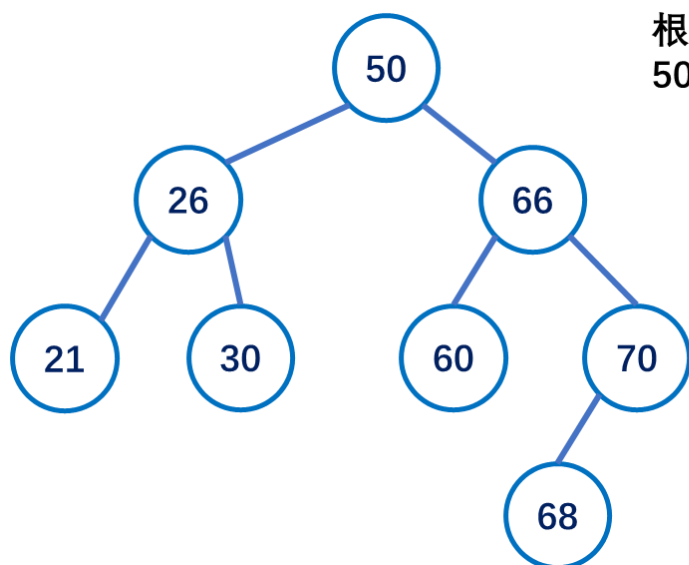
目标：查找关键字为16的结点
 ① $16 < 19$ ，应插入到左子树
 ② $16 > 10$ ，应插入到右子树
 ③ $16 > 13$ ，应插入到右子树
 ④ 13的右子树为空，将要插入的结点16插入到13的右子树

代码实现如下：

```
bool BST_Insert(BSTree &T, int k) {
    if (T == NULL) { // 树为空，此时插入的结点为根节点
        T = (BSTree) malloc(sizeof(BSTNode));
        T->key = k;
        T->lchild = T->rchild = NULL;
        return true;
    } else if (k == T->key) { // 树中村赞关键字相同的结点，插入失败
        return false;
    } else if (k < T->key) { // 插入T的左子树
        return BST_Insert(T->lchild, k);
    } else { // 插入T的右子树
        return BST_Insert(T->rchild, k);
    }
}
```

最坏时间复杂度： $O(h)$ (h 为树高)

4、二叉排序树的构造——不断插入新结点的过程



根据序列构造二叉树：
 50 66 60 26 21 30 70 68

代码实现：

```

void Create_BST(BSTree &T, int str[], int n) {
    T = NULL;
    int i = 0;
    while (i < n) {
        BST_Insert(T, str[i]);
        i++;
    }
}

```

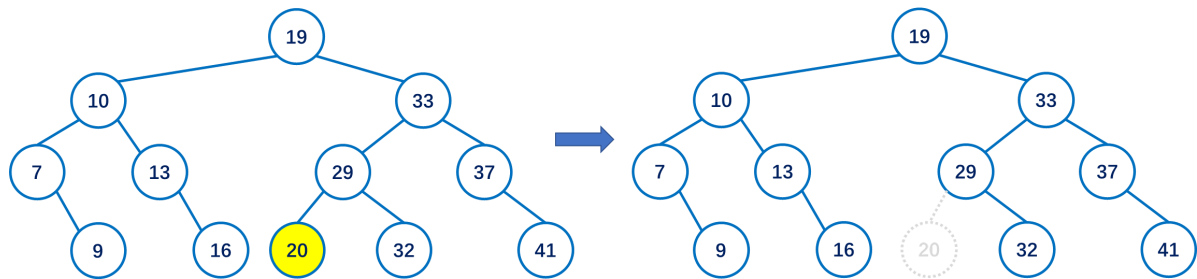
【注意】不同的关键字序列可能得到同款二叉排序树，也可能得到不同的二叉排序树。

5、二叉排序树的删除

先搜索到目标结点，此后有两种情况：

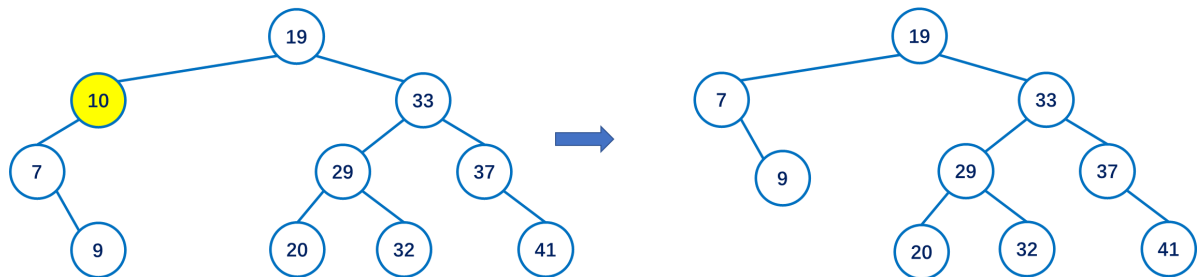
(1) 若被删除的结点 z 是叶子结点，则直接删除，不会破坏二叉排序树的性质。（左子树结点值 $<$ 根节点值 $<$ 右子树结点值）

删除关键字为20的结点（叶节点）



(2) 若被删除的结点 z 只有一棵左子树或右子树，则让 z 的子树称为 z 的父结点的子树，替代 z 的位置。

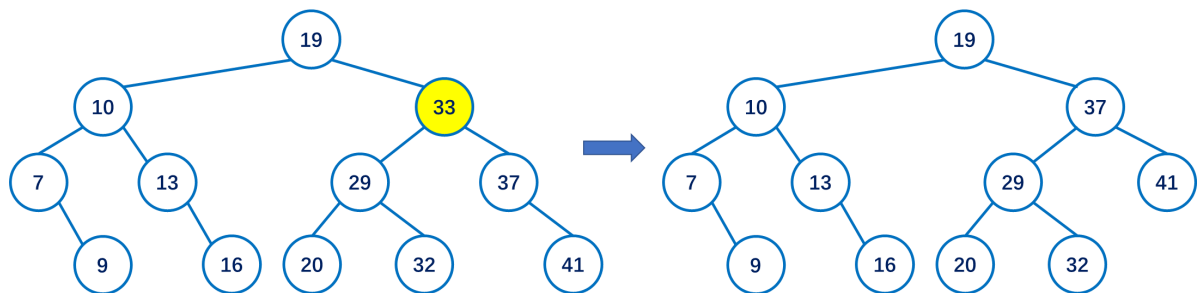
删除关键字为10的结点（只有左子树）



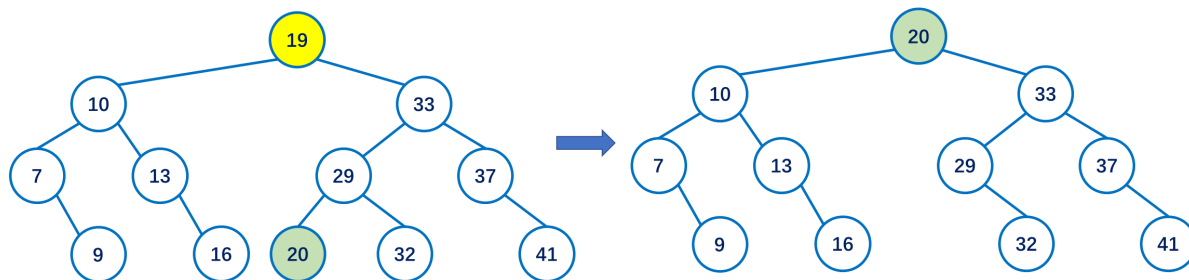
(3) 若被删除的结点 z 有左右两颗子树，则令 z 的直接后继（或直接前驱）替代 z ，然后从二叉排序树中删除这个直接后继（或直接前驱），这样就转化成了第一种或第二种情况。

这里提到的直接前驱和直接后继指的是中序遍历序列的前驱和后继。

删除关键字为33的结点

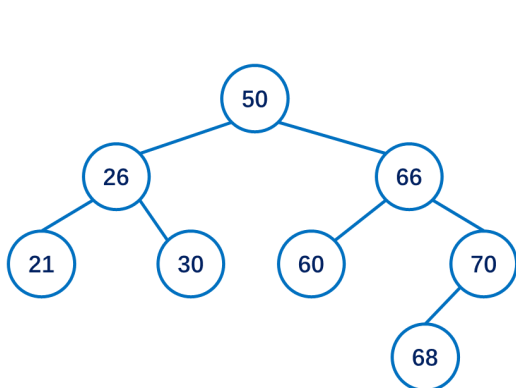


删除关键字为19的结点



6、查找效率分析

查找长度：在查找运算中，需要对比关键字的次数称为查找长度，反映了查找操作时间复杂度

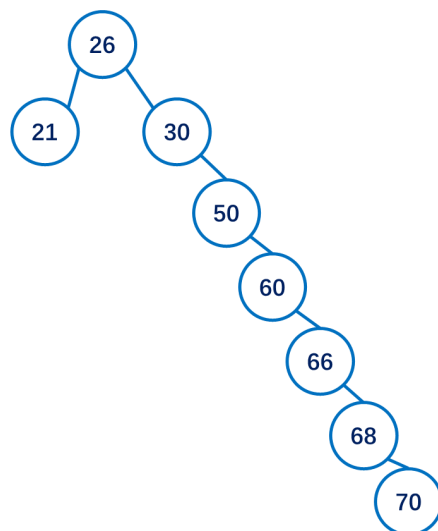


查找成功的平均查找长度：

$$ASL = (1*1 + 2*2 + 3*4 + 4*1) / 8 = 2.675$$

查找失败的平均查找长度：

$$ASL = (3*7 + 4*2) / 9 = 3.22$$



查找成功的平均查找长度：

$$ASL = (1*1 + 2*3 + 3*1 + 4*1 + 5*1 + 6*1 + 7*1) / 8 = 3.75$$

查找失败的平均查找长度：

$$ASL = (2*3 + 3*1 + 4*1 + 5*1 + 6*1 + 7*1) / 9 = 4.22$$

这样的二叉排序树可能会出现右图这种情况，影响排序效率。

最好情况： n 个结点的二叉树最小高度为 $\lceil \log_2 n \rceil + 1$ ，平均查找长度为 $O(\log_2 n)$

最坏情况：每个结点只有一个分支，树高 $h = \text{结点数}n$ ，平均查找长度 $=O(n)$

7.3.2 平衡二叉树 (AVL)

1、定义

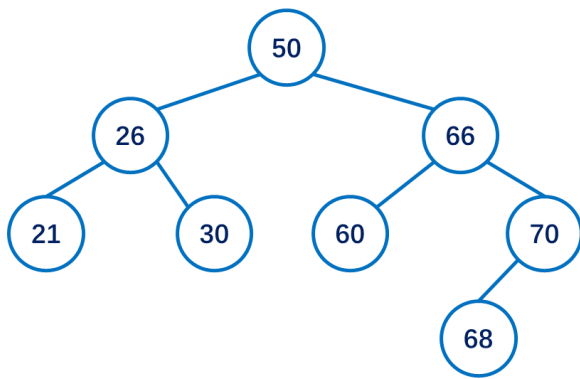
平衡二叉树 (Balanced Binary Tree)，简称平衡树 (AVL树)

树上任一结点的左子树和右子树的高度之差不超过1。

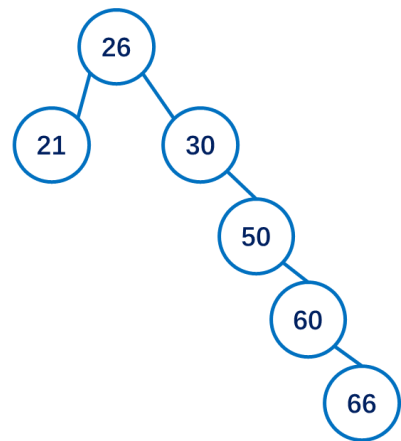
结点的平衡因子=左子树高-右子树高。

平衡因子的值只能为0,1,-1

只要有任一结点的平衡因子的绝对值大于1，就不是平衡二叉树



平衡二叉树
平衡因子=1



非平衡二叉树
平衡因子=-3

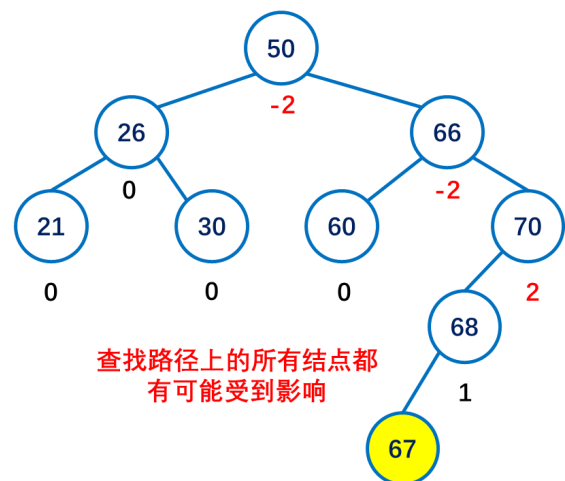
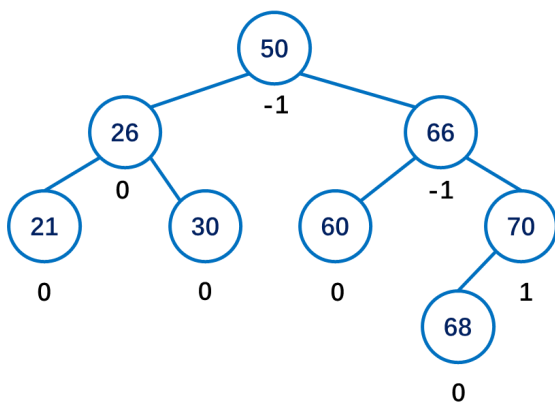
其结构定义如下：

```

typedef struct AVLNode {
    int key;           // 数据域
    int balance;       // 平衡因子
    struct AVLNode *lchild, *rchild;
} AVLNode, *AVLTree;
  
```

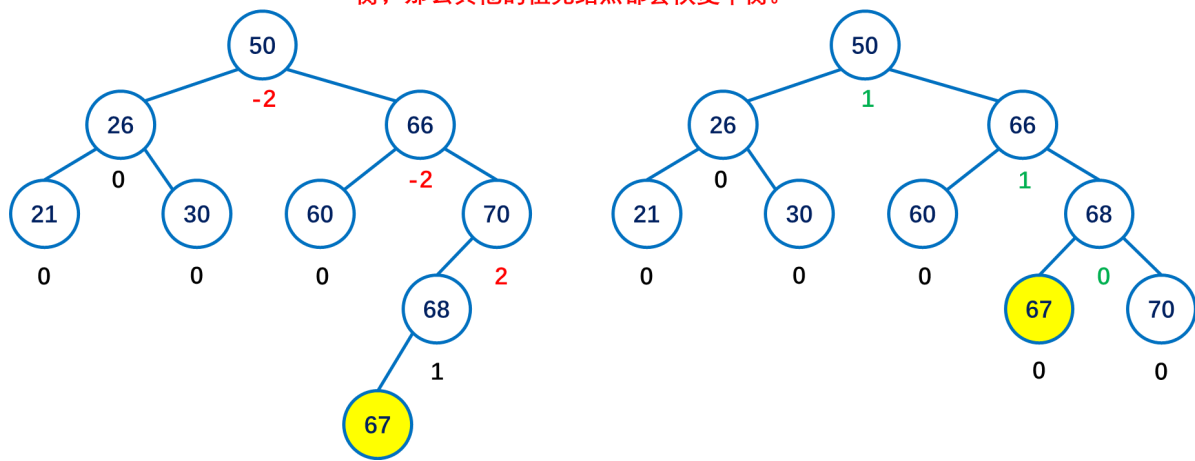
2、平衡二叉树的插入

在平衡的二叉排序树中直接插入结点，使其“不平衡”了



解决方法：通过调整“最小不平衡子树”来再次达到平衡!!!

在插入操作中，只要将“最小不平衡子树”调整平衡，那么其他的祖先结点都会恢复平衡。



根据插入情况的不同，有如下四种情况：

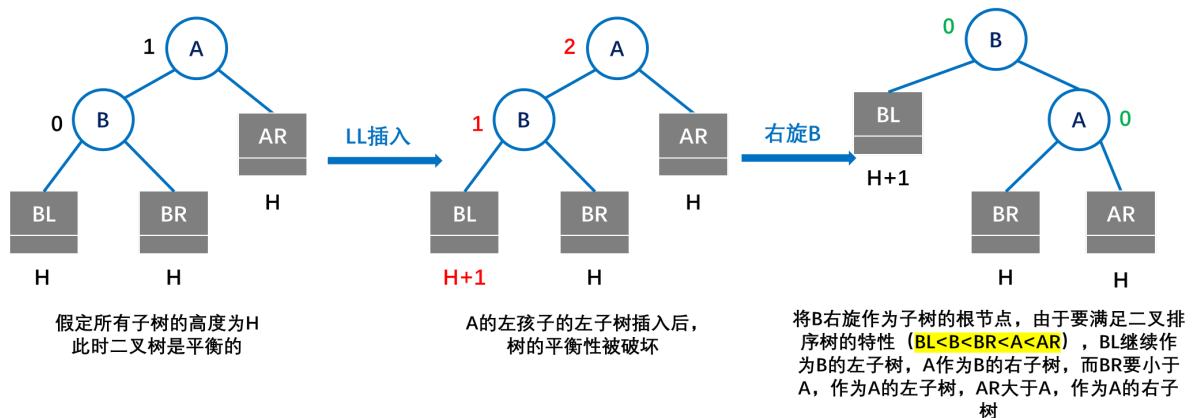
- **LL**：在A的左孩子的左子树中插入导致不平衡
- **RR**：在A的右孩子的右子树中插入导致不平衡
- **LR**：在A的左孩子的右子树中插入导致不平衡
- **RL**：在A的右孩子的左子树中插入导致不平衡

(1) LL平衡旋转

假定所有的子树的高度都是H，是为了在插入节点后，导致树的平衡性被破坏。

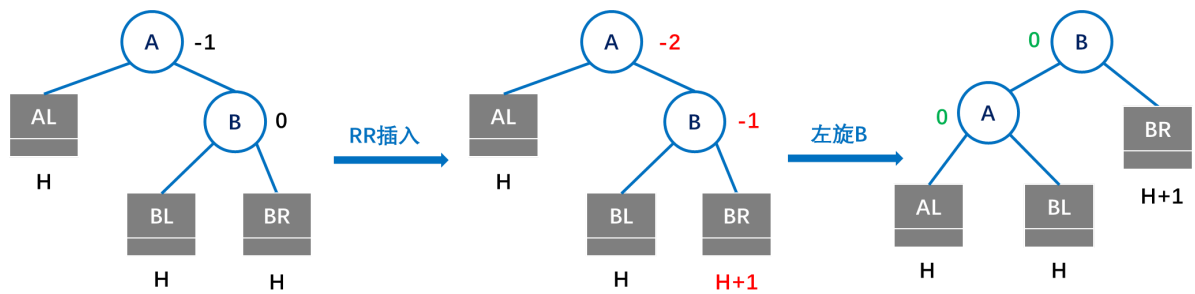
目标：1、恢复平衡；2、保持二叉排序树特性。

操作：右单旋转：由于在结点A的左孩子(L)的左子树(L)上插入了新结点，A的平衡因子由1增至2，**导致以A为根的子树失去平衡**，需要一次**向右的旋转操作**。将A的左孩子B向右上旋转代替A成为根结点，将A结点向右下旋转成为B的右子树的根结点，而B的原右子树则作为A结点的左子树。



(2) RR平衡旋转

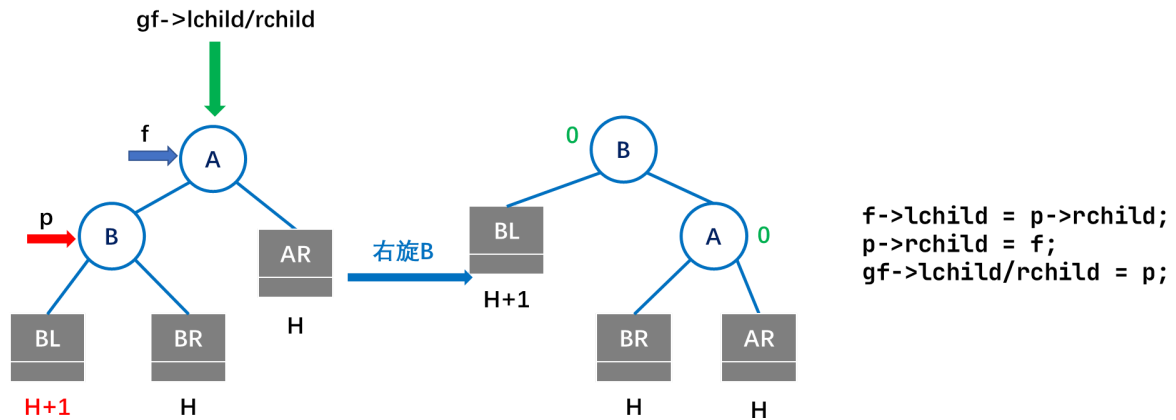
操作：RR平衡旋转（左单旋转）：由于在结点A的右孩子(R)的右子树(R)上插入了新结点，A的平衡因子由-1减至-2，导致以A为根的子树失去平衡，需要一次**向左的旋转操作**。将A的右孩子B向左上旋转代替A成为根结点，将A结点向左下旋转成为B的左子树的根结点，而B的原左子树则作为A结点的右子树



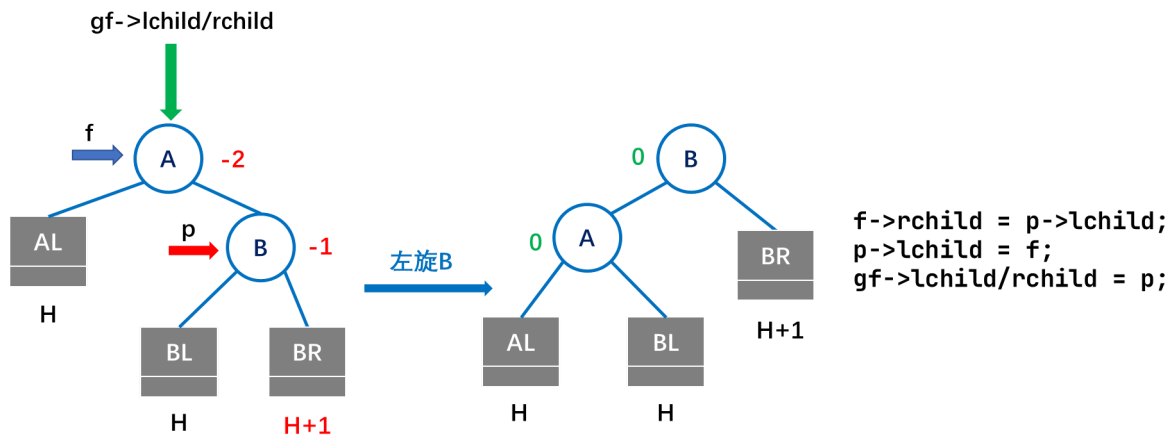
把B左旋作为最小不平衡子树的根节点，由于要满足二叉排序树的特性 ($AL < A < BL < B < BR$)，故BR仍是B的右子树，AL仍是A的左子树，而BL需要小于B大于A，作为A的右子树。

(3) 代码思路

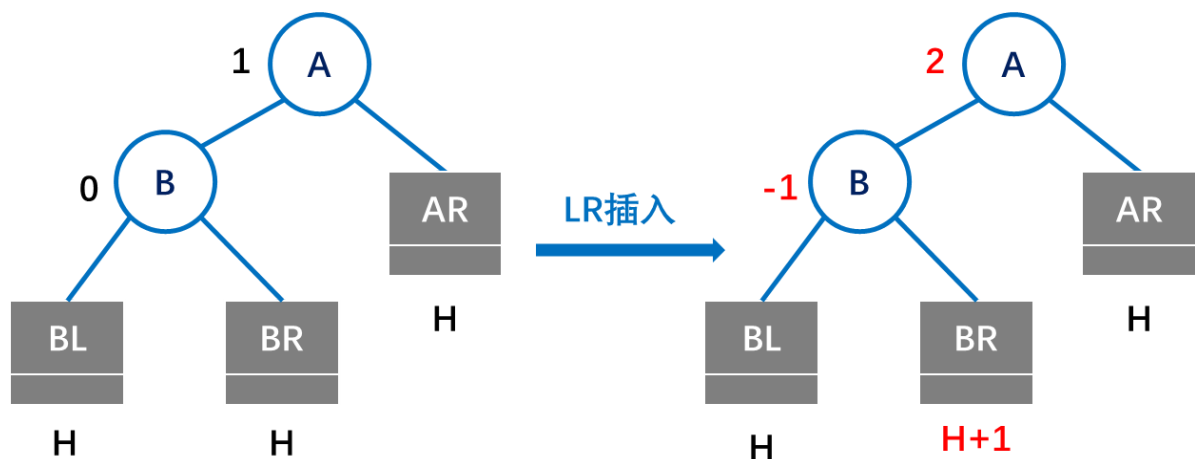
实现f向右下旋转，p向右上旋转。



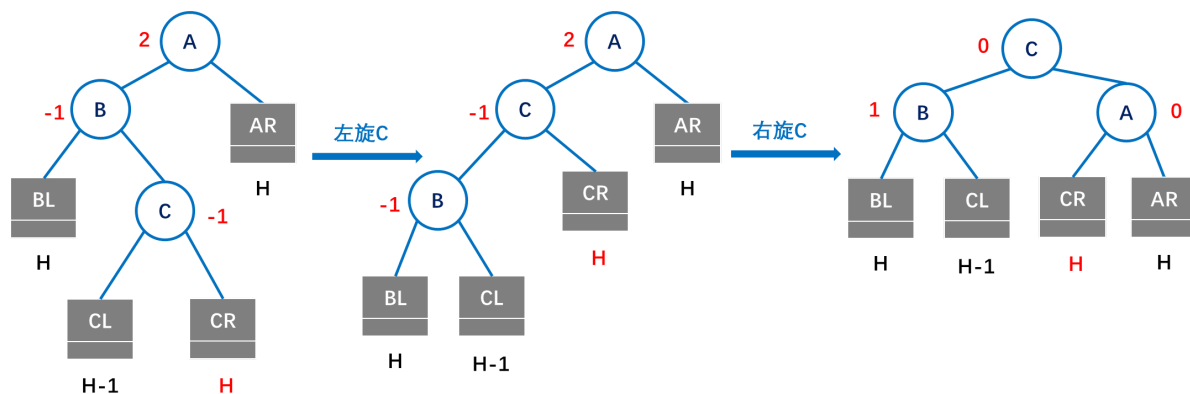
实现f向左下旋转，p向左上旋转。



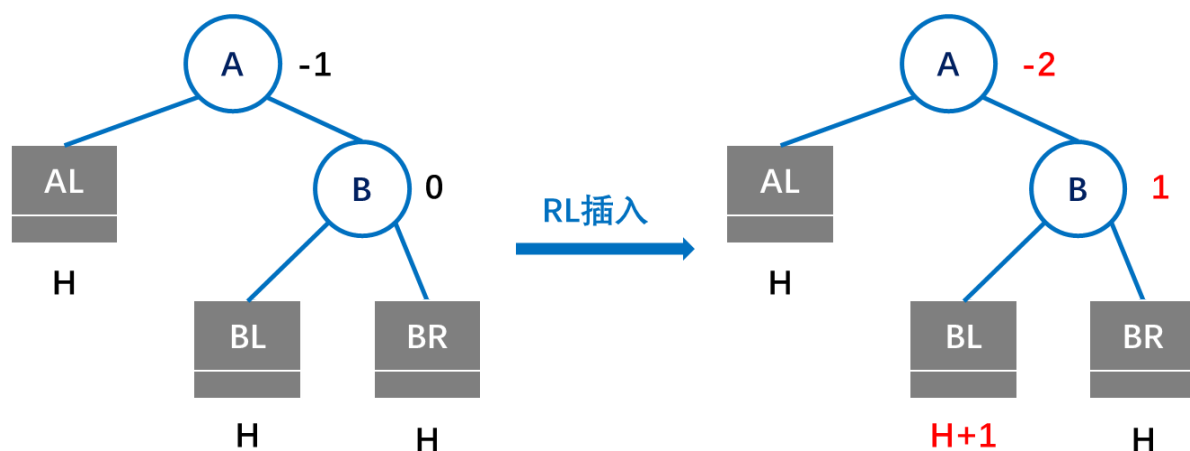
(4) LR平衡旋转



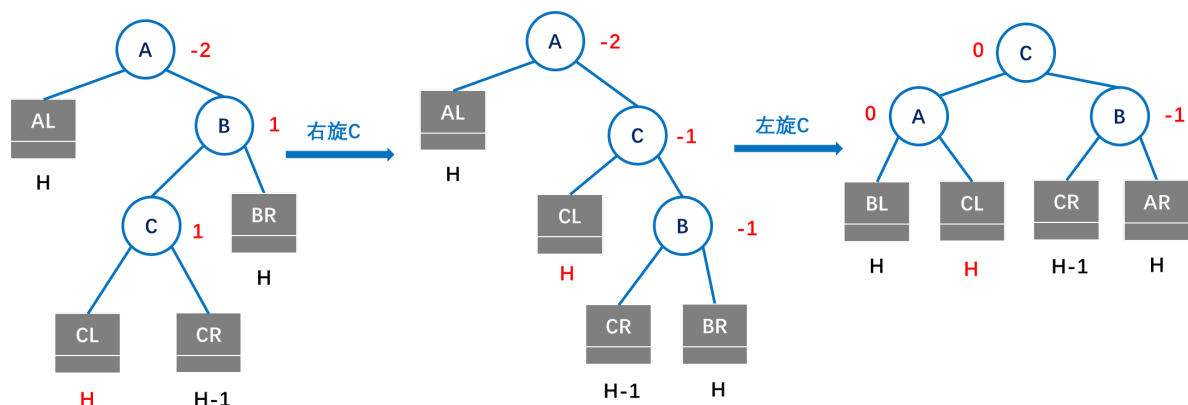
先左后右双旋转：由于在A的左孩子(L)的右子树(R)上插入新结点，A的平衡因子由1增至2，导致以A为根的子树失去平衡，需要进行两次旋转操作，**先左旋转后右旋转**。先将A结点的左孩子B的右子树的根结点C向左上旋转提升到B结点的位置，然后再把该C结点向右上旋转提升到A结点的位置



(5) RL平衡旋转



先右后左双旋转：由于在A的右孩子(R)的左子树(L)上插入新结点，A的平衡因子由-1减至-2，导致以A为根的子树失去平衡，需要进行两次旋转操作，先右旋转后左旋转。先将A结点的右孩子B的左子树的根结点C向右上旋转提升到B结点的位置，然后再把该C结点向左上旋转提升到A结点的位置



总结：

- **LL**：在A的**左孩子**的**左子树**中插入导致不平衡
 - 调整：A的左孩子节点右上旋
- **RR**：在A的**右孩子**的**右子树**中插入导致不平衡
 - 调整：A的右孩子节点右上旋
- **LR**：在A的**左孩子**的**右子树**中插入导致不平衡
 - 调整：A的左孩子的右孩子节点 先左上旋后右上旋
- **RL**：在A的**右孩子**的**左子树**中插入导致不平衡

- 调整：A的右孩子的左孩子节点 先右上旋后左上旋

3、查找效率分析

若树高为 h ，则最坏情况下，查找一个关键字最多需要对比 h 次，即查找的时间复杂度不超过 $O(h)$

【记忆】平衡二叉树的平均查找长度为 $O(\log_2 n)$ (证明十分麻烦，记住即可)

4、平衡二叉树的删除

平衡二叉树的删除操作具体步骤：

- 删除结点（方法同“二叉排序树”）
 - 若删除的结点是叶子，直接删。
 - 若删除的结点只有一个子树，用子树顶替删除位置
 - 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱(或后继)结点的删除
- 向上找到最小不平衡子树，找不到则说明已经平衡。
- 找最小不平衡子树下，“个头”最高的儿子、孙子
- 根据孙子的位置，调整平衡(LL/RR/LR/RL)
 - 孙子在LL:儿子右单旋
 - 孙子在RR:儿子左单旋
 - 孙子在LR:孙子先左旋，再右旋
 - 孙子在RL:孙子先右旋，再左旋
- 如果不平衡向上传导，继续第二步
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）

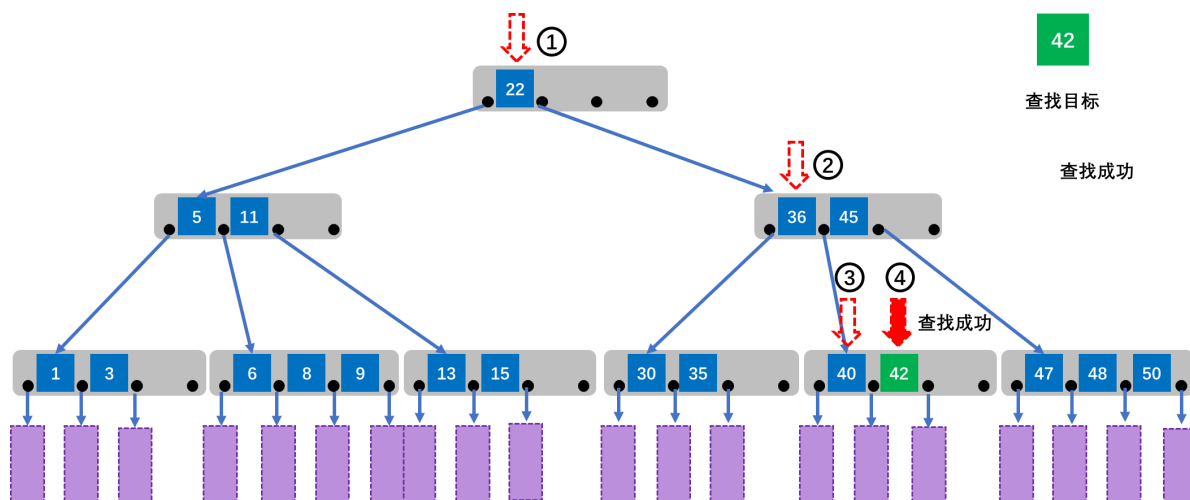
平衡二叉树删除的时间复杂度： $O(\log_2 n)$

7.4 B树

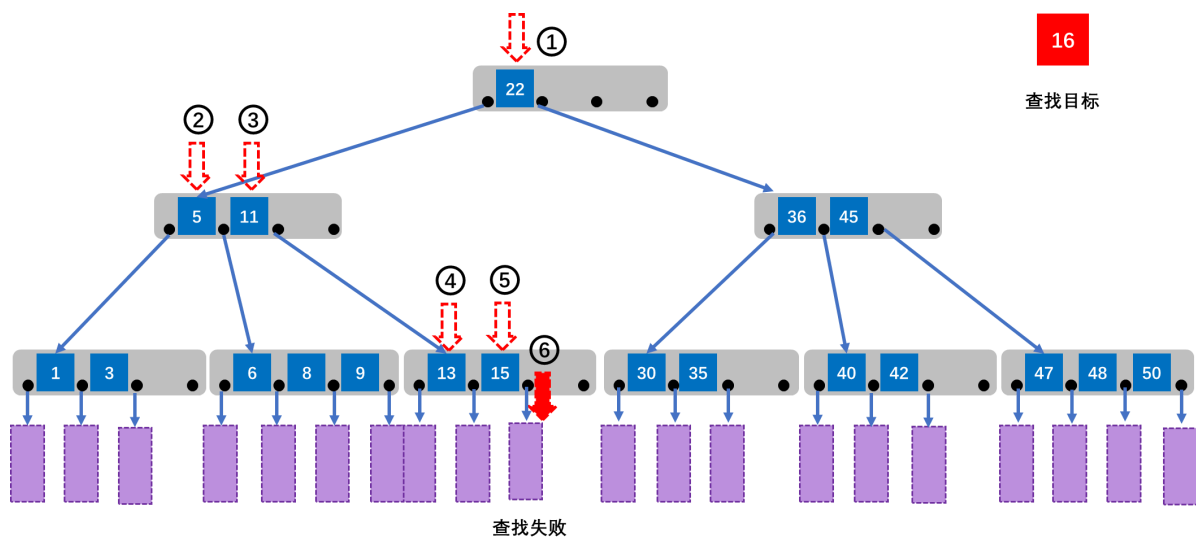
7.4.1 B树及其基本操作

1、m叉查找树

查找成功的情况：



查找失败的情况

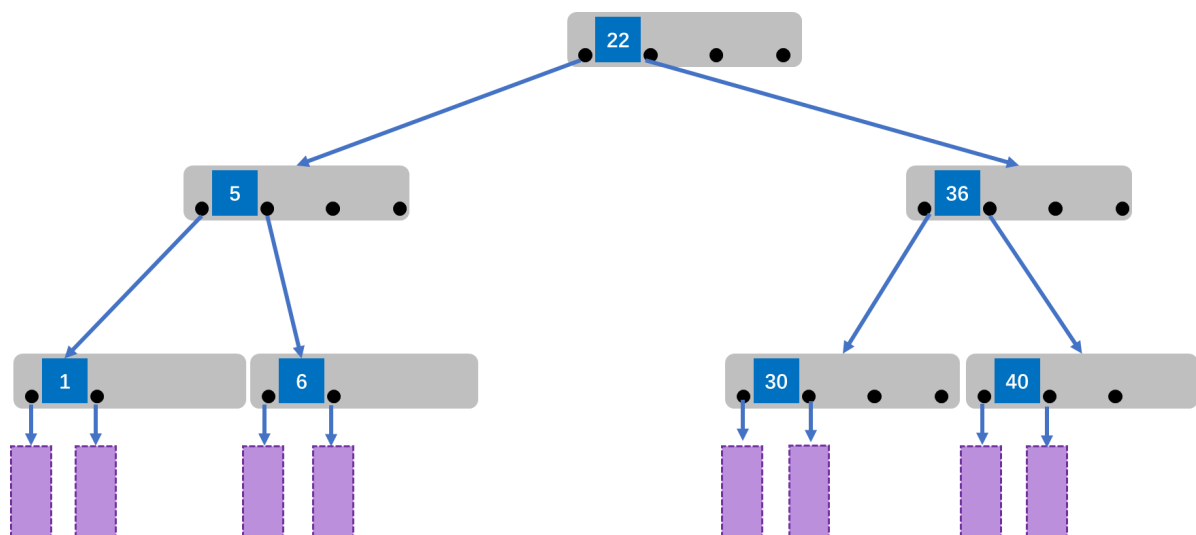


2、如何保证查找效率

规定：

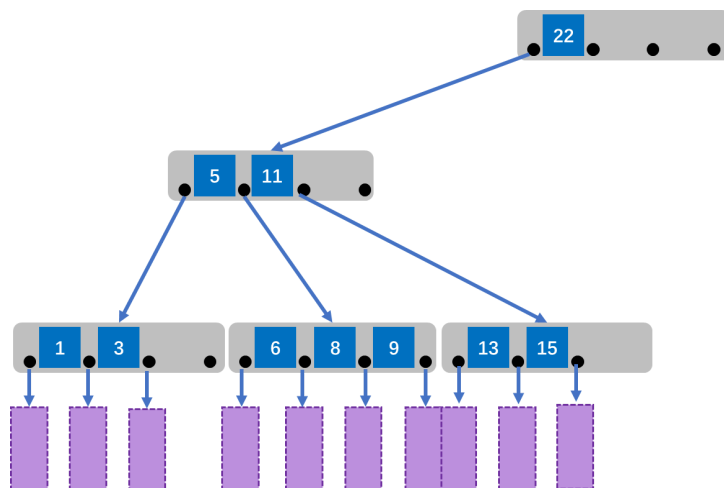
- m 叉查找树中，除了根节点外，任何结点至少有 $\lceil m/2 \rceil$ 个分叉，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字。

否则会很稀疏，浪费空间



- m 叉查找树中，对于任何一个结点，其所有子树的高度要相同。

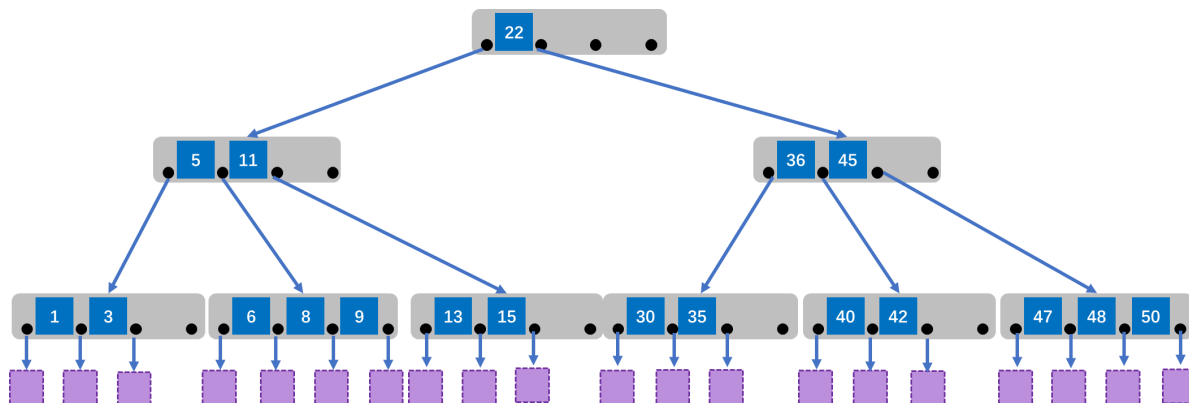
否则，树高过高，需要查询很多层结点



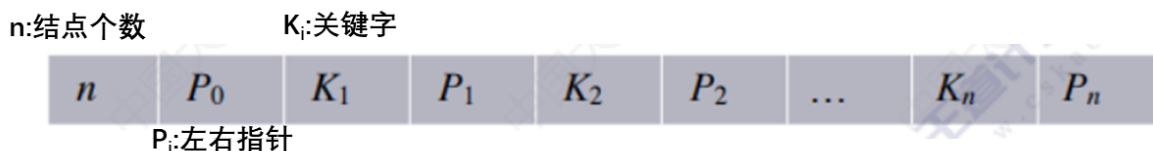
3、B树定义

B树，又称**多路平衡查找树**，B树中所有结点的孩子个数的最大值称为B树的阶，通常用m表示。一棵m阶B树或为空树，或为满足如下特性的m叉树：

1. 树中每个结点至多有m棵子树，即至多含有m-1个关键字。
2. 若根结点不是终端结点，则至少有两棵子树。
3. 除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字。
4. 所有的叶结点都出现在同一层次上，并且不带信息（可以视为外部结点或类似于折半查找判定树的查找失败结点，实际上这些结点不存在，指向这些结点的指针为空）。



其结点结构如下：



m阶B树的核心特性：

1. 根节点的子树数 $\in [2, m]$ ，关键字数 $\in [1, m - 1]$
其他节点的子树数 $\in [\lceil m/2 \rceil, m]$ ，关键字数 $\in [\lceil m/2 \rceil - 1, m - 1]$
2. 对任一结点，**其所有子树高度都相同**
3. 关键字的值：子树0<关键字1<子树1<关键字2<子树2<....（类比二叉查找树 左<中<右）

B树的高度：含n个关键字的m阶B树，最小高度、最大高度为多少？

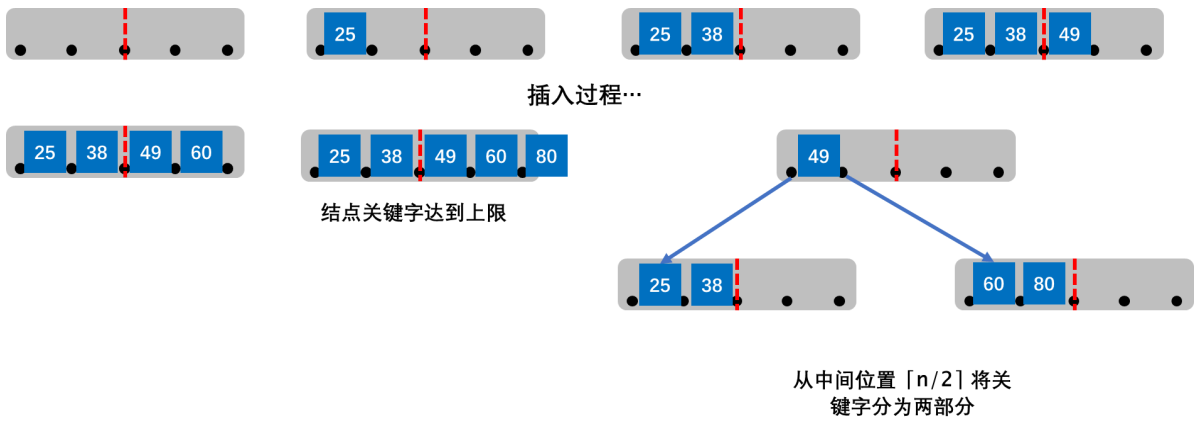
$$\log_m(n + 1) \leq h < \log_{\lceil m/2 \rceil} \frac{m + 1}{2} + 1$$

7.4.2 B树的基本操作

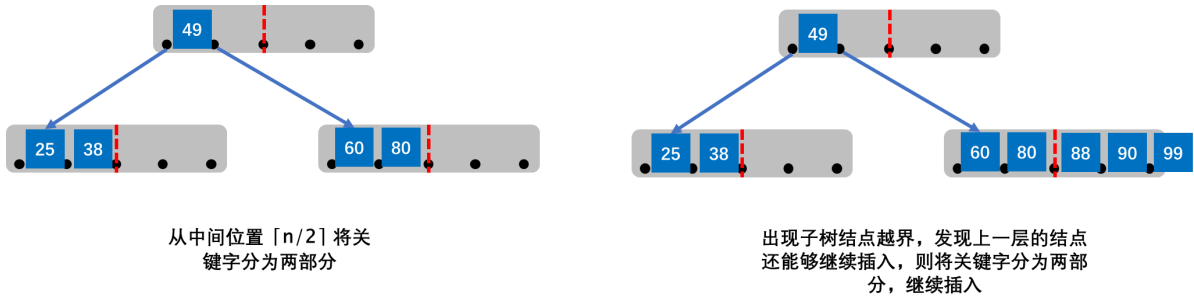
1、B树的插入

如下示例（此处省略失败节点）：

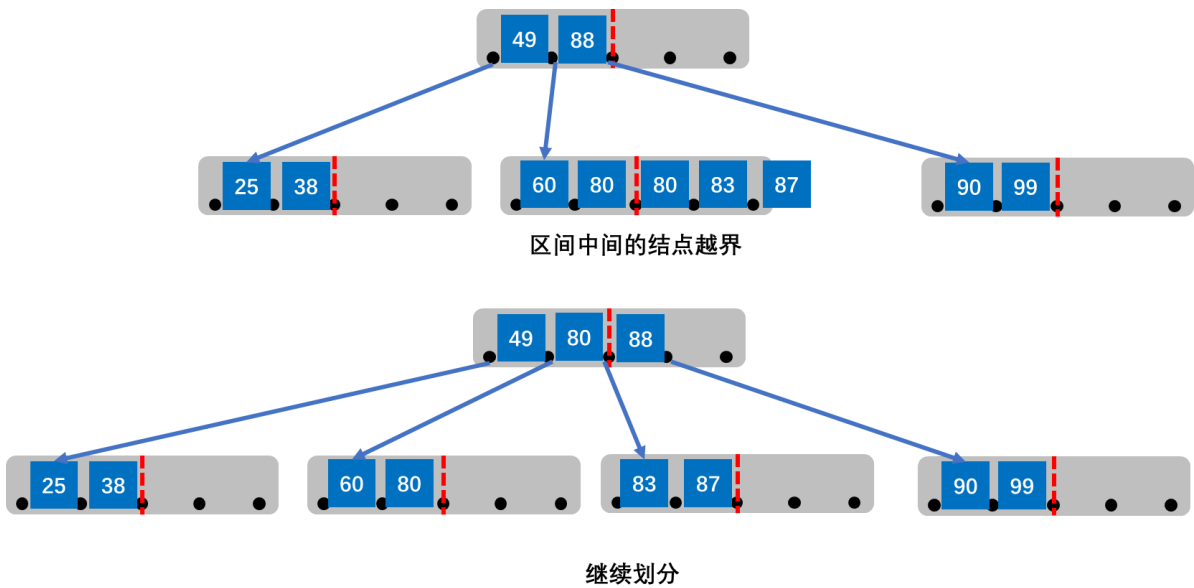
情形1：插入根节点



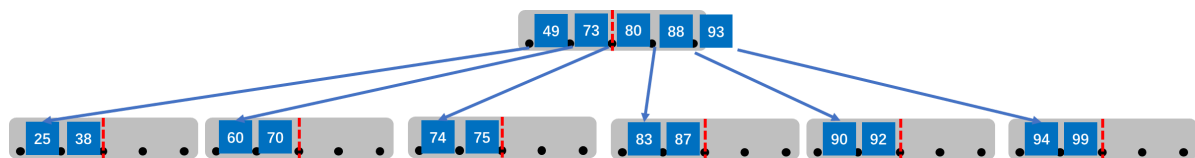
情形2:



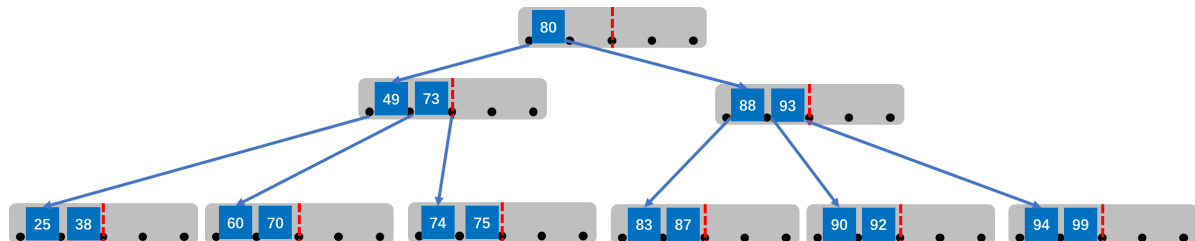
情形3:



情形4:



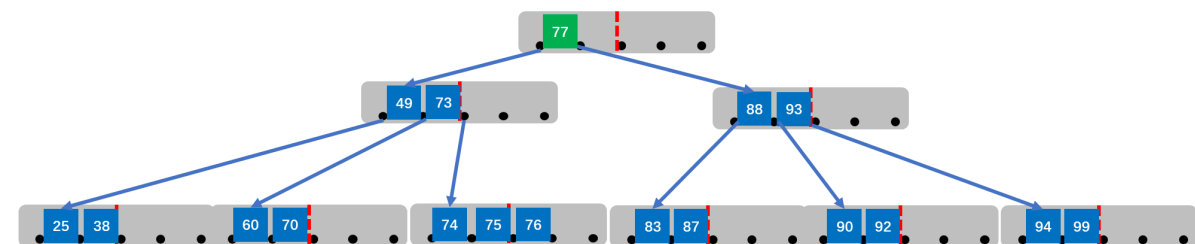
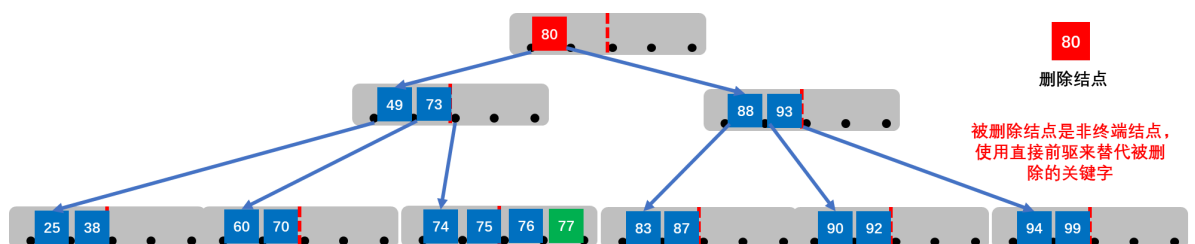
原结点插入导致越界，进行划分，导致父结点越界



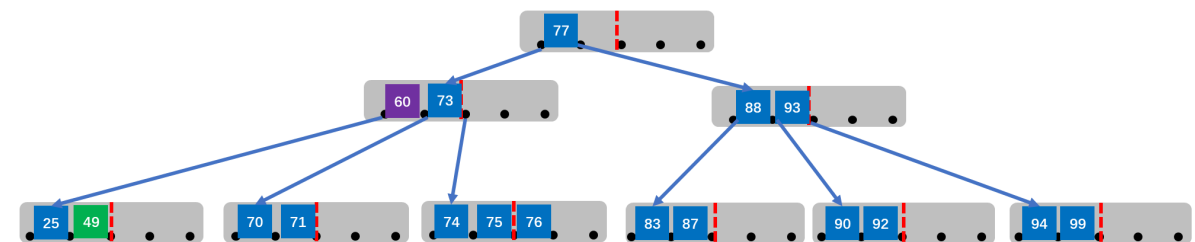
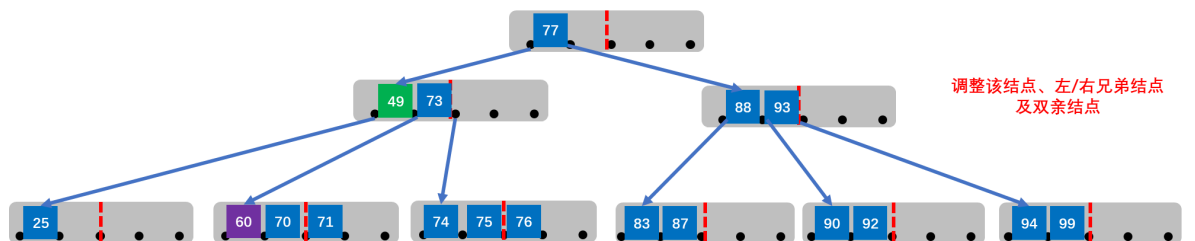
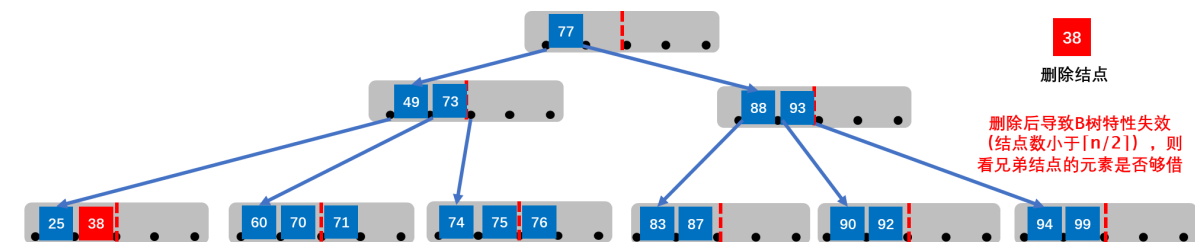
对父结点进行分裂操作，直至这个过程传到根节点为止，这会导致B树高度增加1

2、B树的删除

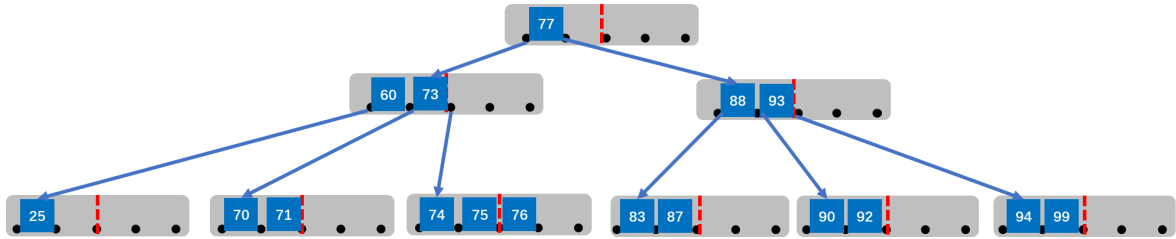
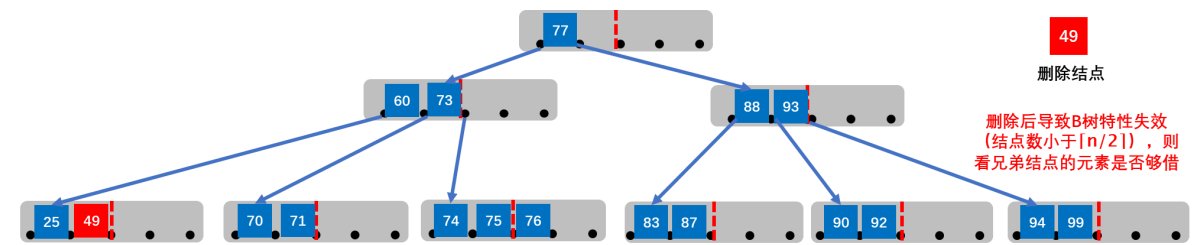
情形1：被删除结点是非终端结点，使用直接前驱或直接后继来替代被删除的关键字



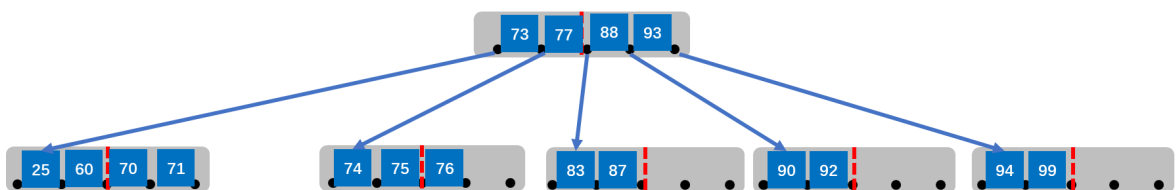
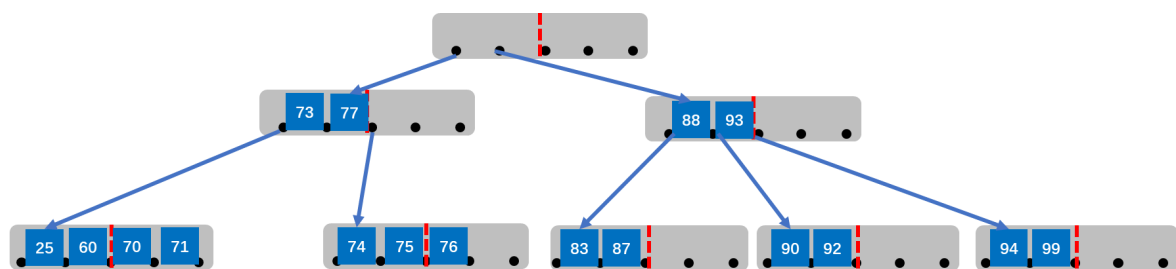
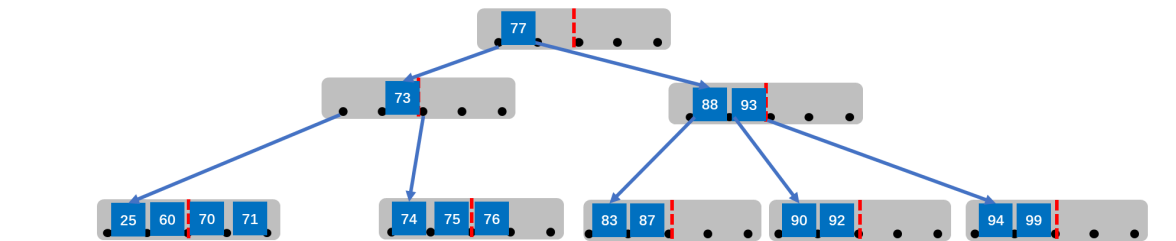
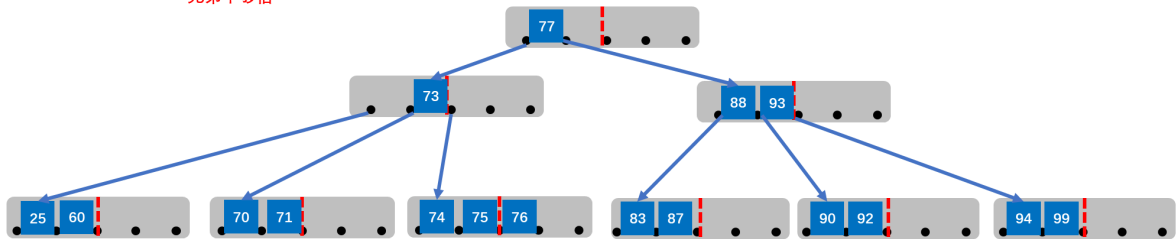
情形2：删除后导致B树特性失效（结点数小于 $\lceil n/2 \rceil$ ），则看兄弟结点的元素是否够借



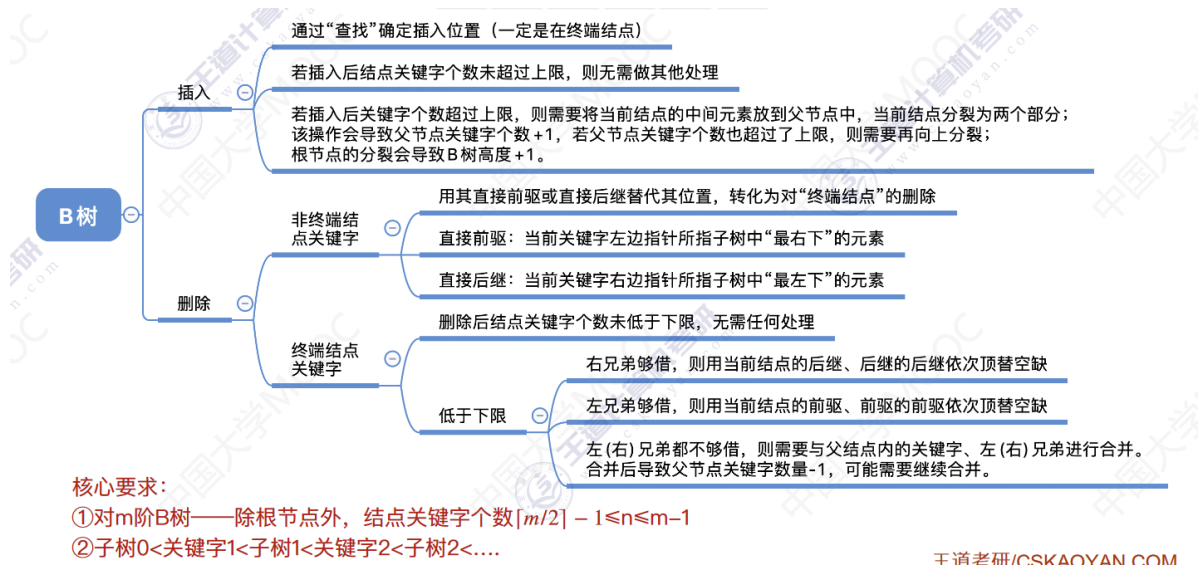
情形3：兄弟结点不够借，将关键字删除后，与左（或右）兄弟结点及双亲结点中的关键字合并



兄弟不够借



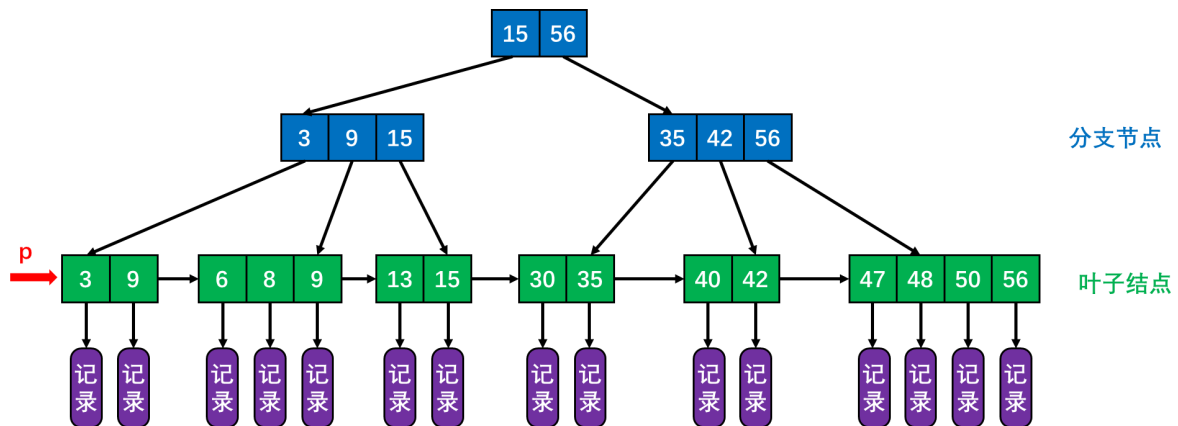
总结：



7.5 B+树

1、定义

下图是一颗4阶B+树：

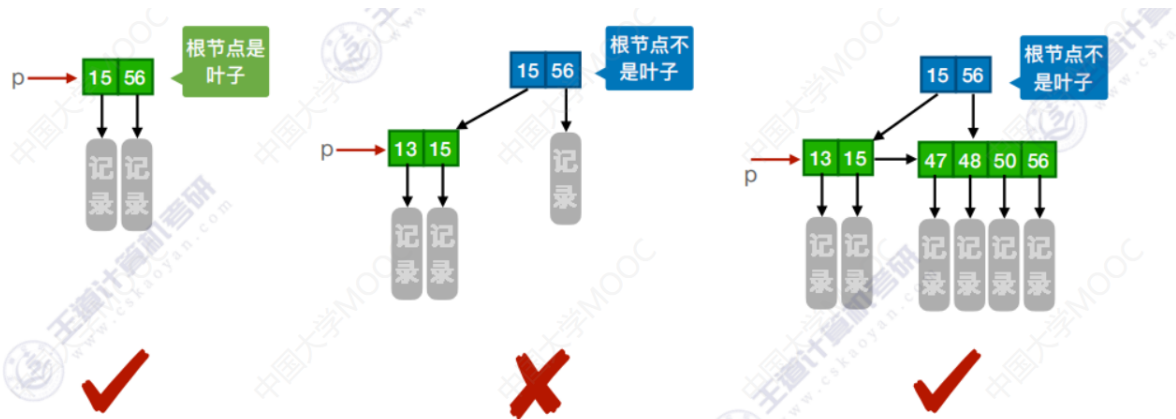


一颗m阶的B+树需要满足以下条件：

1. 每个分支节点最多有m棵子树（孩子结点）
2. 非叶根节点至少有两颗子树，其他每个分支节点至少有 $\lceil n/2 \rceil$ 棵子树
3. 结点的子树个数与关键字个数相等
4. 所有叶节点包含全部关键字及指向相应记录的指针，叶节点中将关键字按照大小顺序排列，并且相邻叶节点按大小顺序相互链接起来。

支持顺序查找

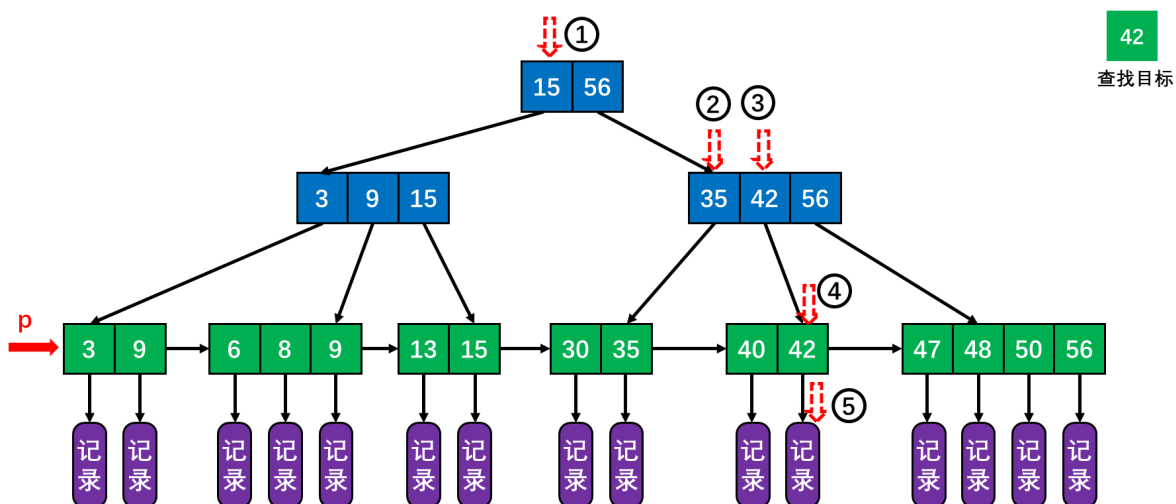
5. 所有分支结点中仅包含它的各个子节点中关键字的最大值及指向其子节点的指针。



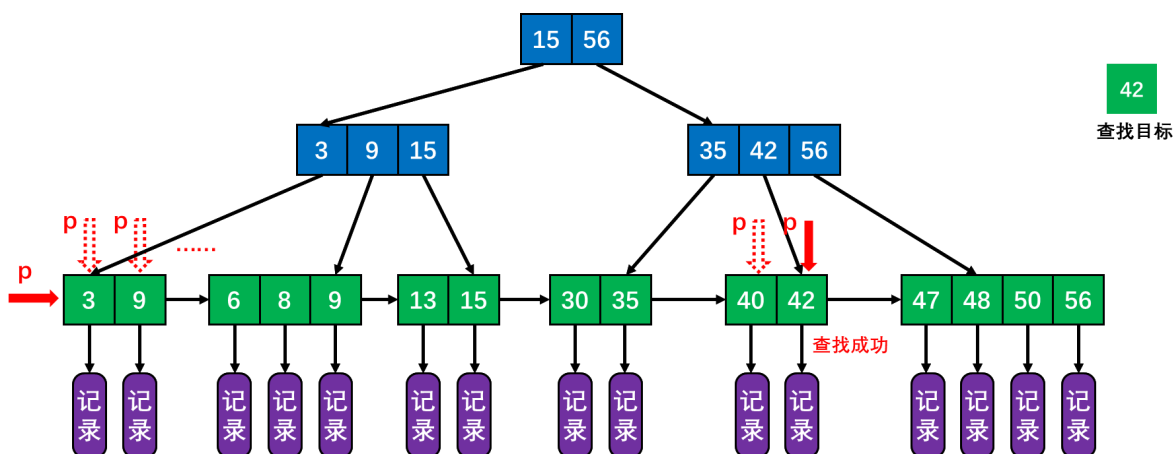
可以理解为：要追求“绝对平衡”，即所有子树高度要相同

2、B+树的查找

多路查找



顺序查找



在B+树中，非叶节点不包含关键字对应记录的存储地址，这样可以使一个磁盘块中包含更多个关键字，使得B+树的阶更大，树高更矮，读磁盘次数更少，查找更快。

3、B树与B+树

	m阶B树	m阶B+树
类比	二叉查找树的进化-->m叉查找树	分块查找的进化-->多级分块查找
关键字与分叉	n个关键字对应n+1个分叉(子树)	n个关键字对应n个分叉
节点包含的信息	所有结点都包含记录的信息	只有最下层叶节点才包含记录的信息
查找方式	不支持顺序查找, 查找成功时, 可能会停留在任何一层结点, 查找速度“不稳定”	支持顺序查找。查找成功或失败都会到达最下一层结点, 查找速度“稳定”
相同点	除了根节点意外, 最少 $\lceil n/2 \rceil$ 个分叉 (确保结点不要太空), 任何一个结点的子树都要一样高 (确保绝对平衡)。	除了根节点意外, 最少 $\lceil n/2 \rceil$ 个分叉 (确保结点不要太空), 任何一个结点的子树都要一样高 (确保绝对平衡)。

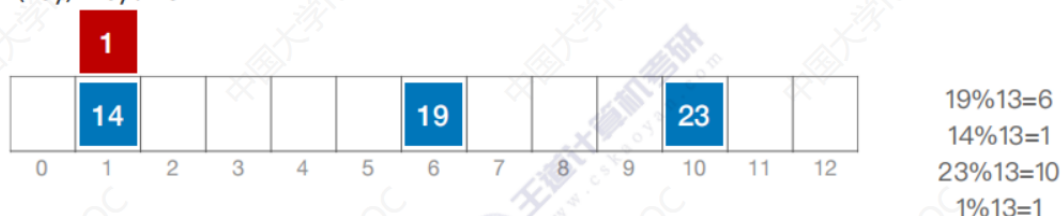
7.6 散列查找

7.6.1 散列表

散列表, 又叫哈希表, 是一种数据结构。其特点是: **数据元素的关键字与其存储地址直接相关。**

- 如何建立“关键字”与“存储地址”的关系?
- 通过**散列函数** (哈希函数) : $Addr = H(key)$

例: 有一堆数据元素, 关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}, 散列函数 $H(key)=key\%13$



若不同的关键字通过散列函数映射到同一个值, 则称它们为“**同义词**”
通过散列函数确定的位置已经存放了其他元素, 则称这种情况为“**冲突**”

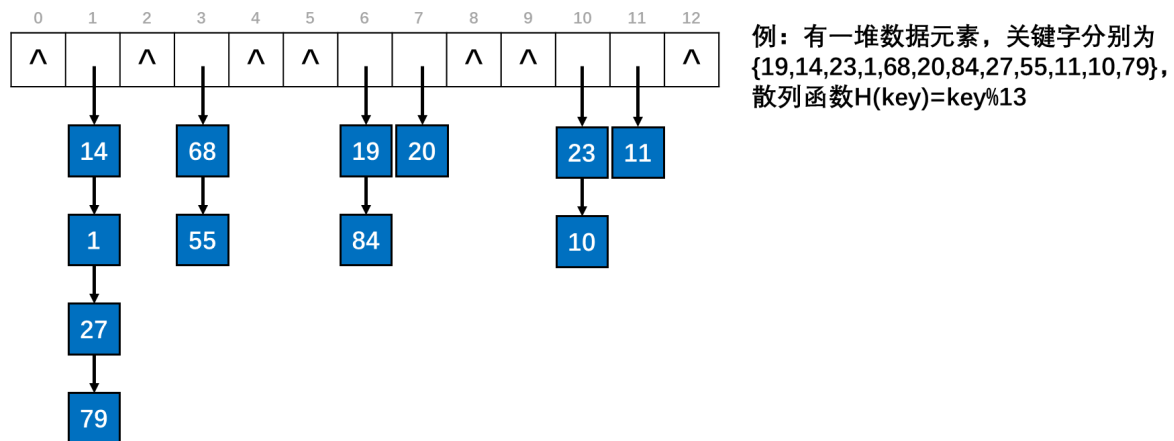
若不同的关键字通过散列函数映射到同一个值, 则称它们为“**同义词**”

通过散列函数确定的位置已经存放了其他元素, 则称这种情况为“**冲突**”

7.6.2 处理冲突的方法

1、拉链法（链地址法）

用拉链法（又称链接法、链地址法）处理“冲突”：**把所有“同义词”存储在一个链表中**



查找方法：

2、开放定址法

所谓开放定址法，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放，其数学递推式为：

$$H_i = (H(key) + d_i) \% m$$

其中， $i = 0, 1, 2, \dots, k (k \leq m - 1)$ ， m 表示**散列表表长**， d_i 为增量序列， i 可以理解为“第 i 次发生冲突”。

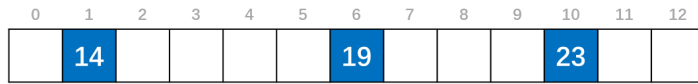
我们需要学习的开放定址法有如下三种：

1. 线性探测法
2. 平方探测法
3. 伪随机探测法

(1) 线性探测法

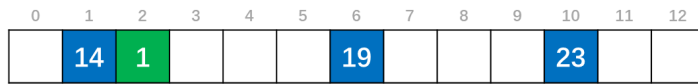
$d_i = 0, 1, 2, 3, \dots, m - 1$ ，即发生冲突时，每次往后探测相邻的下一个单元是否为空。

线性探测法：发生冲突时，每次往后探测相邻的下一个单元是否为空。 $H_i = (H(key) + d_i) \% m$ $d_i=0,1,2,3,...,m-1$

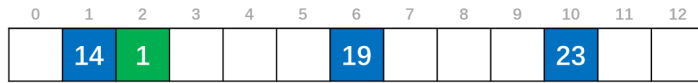


$(1+d_0)\%13=1$ ，发生冲突

1

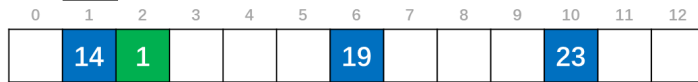


$(1+d_1)\%13=2$
2号单元为空，存入2号单元



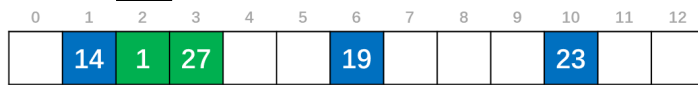
$(27+d_0)\%13=1$ ，发生冲突

27



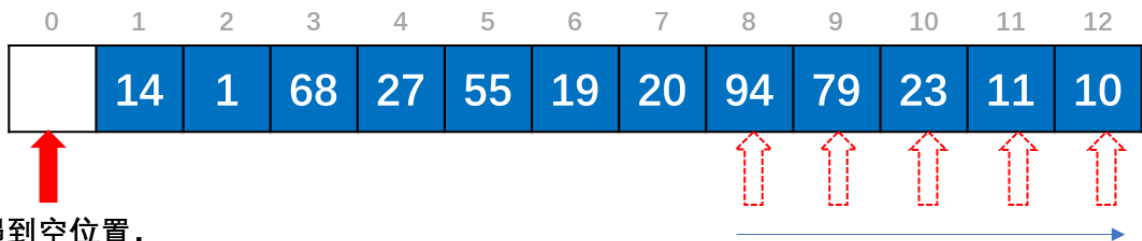
$(27+d_1)\%13=2$ ，依旧冲突

27



$(27+d_2)\%13=3$
3号单元为空，存入3号单元

查找操作：根据计算到的地址，依次往后比对，直到遇见空位置，说明查找失败。



遇到空位置，
查找失败

删除操作：使用这种方法删除元素时，不能简单地将被删除结点的空间置为空，需要做标记，否则会结点它之后填入的散列表的同义词结点的查找路径，可以做一个删除标记，进行逻辑删除。

查找效率分析

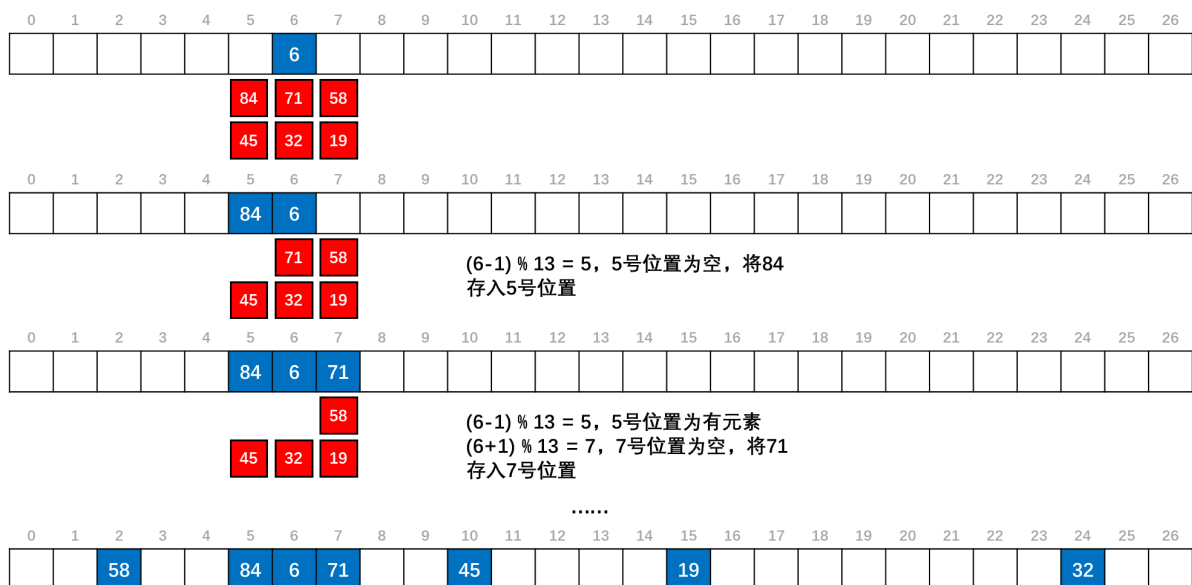
线性探测法操作简单，但是很容易造成同义词、非同义词的“聚集现象”，严重影响查找效率。

产生原因：冲突后再探测一定是放在某个连续的位置。

(2) 平方探测法

$d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2m - k^2$ 时，称为平方探测法，又称二次探测法，其中 $k \leq m/2$

使用平方探测法来处理冲突 $H(key) = key \% 13$



平方探测法相比线性探测法更不容易出现“聚集（堆积）问题”

(3) 伪随机序列法

d_i 是一个伪随机序列, 如 $d_i = 0, 5, 24, 11, \dots$

【注意】采用开放定址法时, 删除结点不能简单地将被删除结点的空间置为空, 否则将截断在他之后填入散列表的同义词的查找路径, 可以做一个删除标记, 进行逻辑删除。

3、再散列法

准备多个散列函数, 一旦冲突, 就用下一个。

4、查找效率

取决于散列函数、处理冲突的方法、装填因子 α 。

5、总结

